

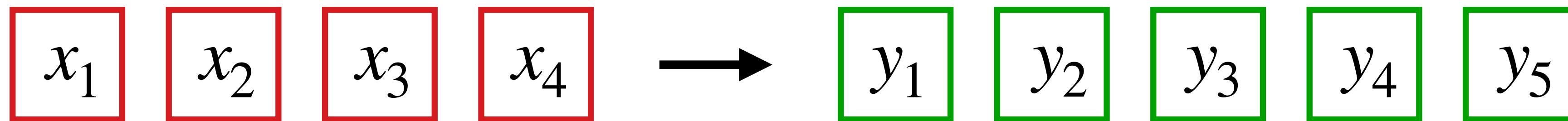
Seq2seq and Transformer

Agenda

- Seq2seq problem statement
- Recurrent Networks for Seq2seq
- Attention Mechanism
- Transformer
- Methods for token decoding

Seq2seq

Convert an input sequence into a new sequence



Sequence lengths may vary

Examples of Seq2seq tasks

1. Machine Translation

Russian ↔ English

2. Summarization

Long text → short text

3. Text style transfer

Rude ↔ Polite

Formal ↔ Informal

Problem statement

Text generation:

$$p(x_1, \dots, x_m) = \prod_{i=1}^m p(x_i | x_{<i})$$

Seq2seq: conditioning on a text

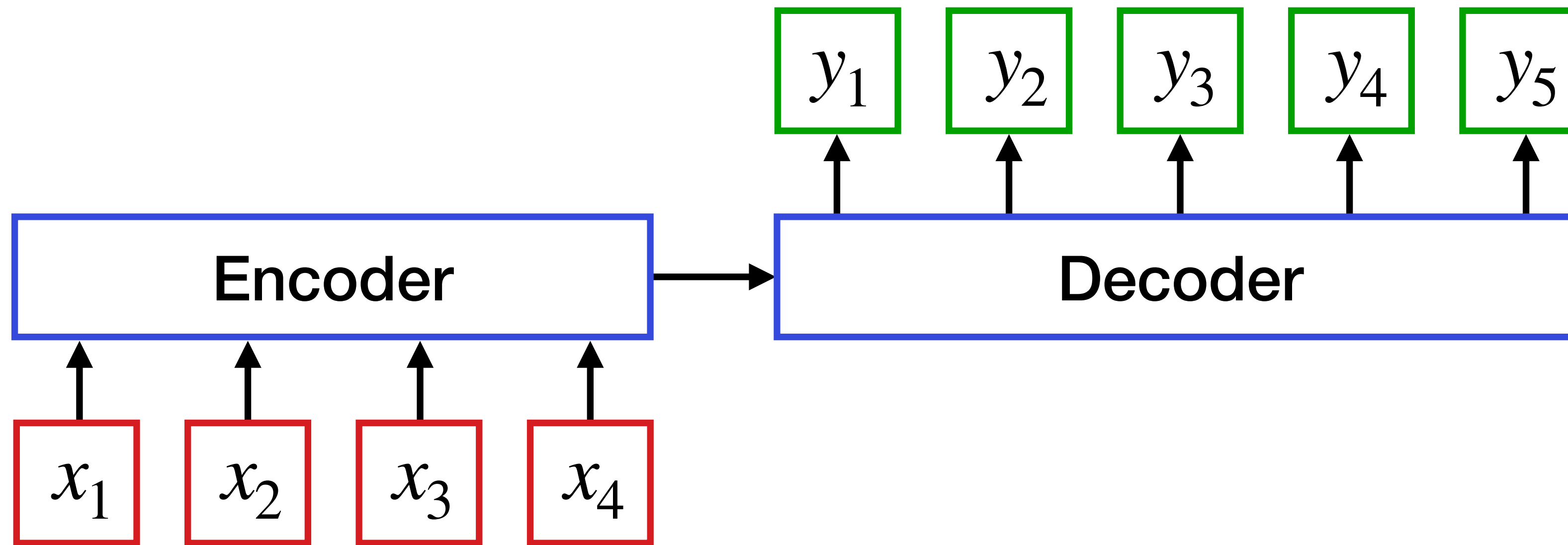
$$p(y_1, \dots, y_n | x_1, \dots, x_m) = \prod_{i=1}^n p(y_i | y_{<i}, x_1, \dots, x_m)$$

x – input sequence

y – output sequence

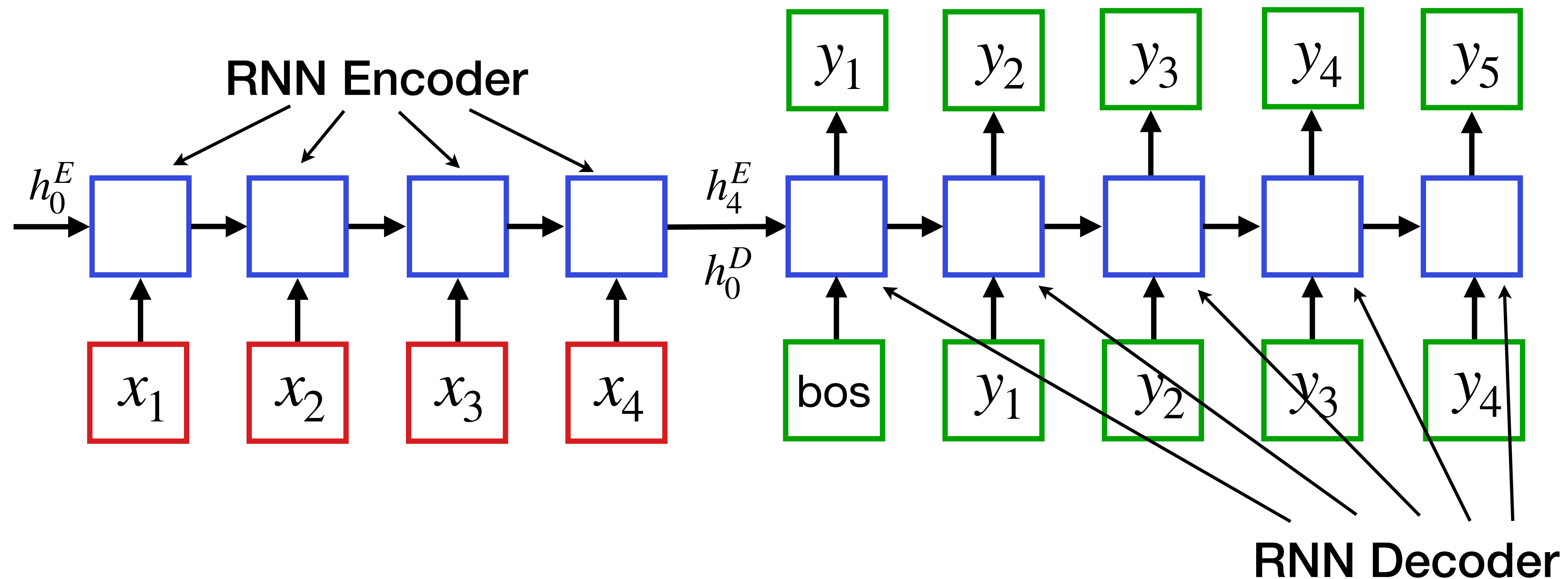
Standard approach

- The input sequence is encoded by the **Encoder** model
- The **Decoder** model takes the output of the **Encoder** and generates a new sequence



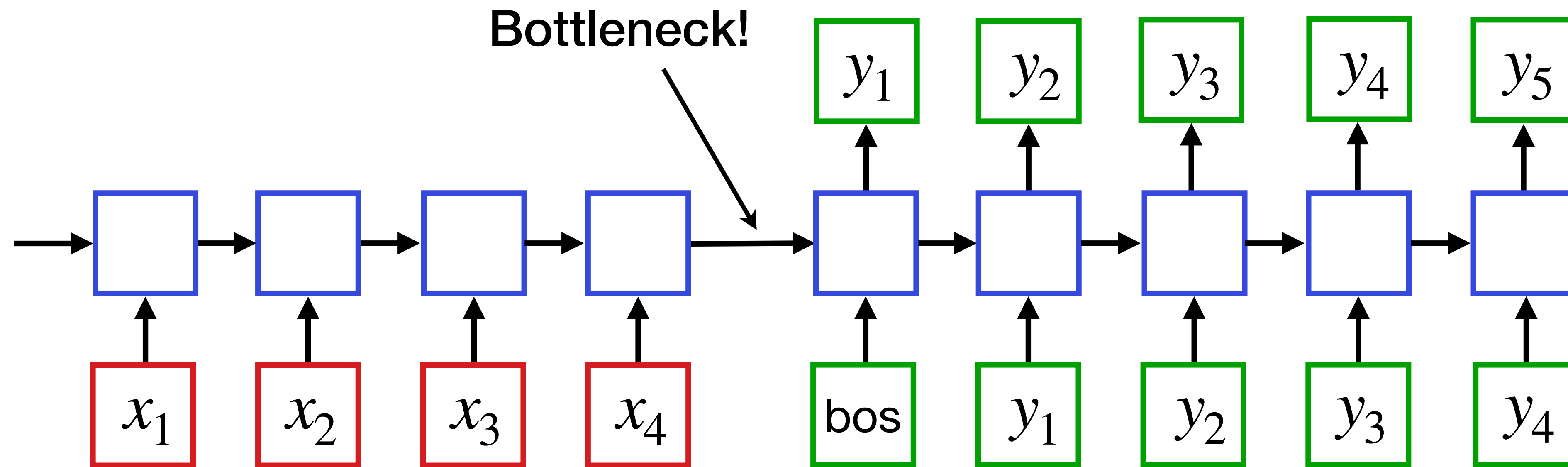
RNN-based approach

- The input sequence is encoded by the **Encoder** model
- The **Decoder** model takes the output of the **Encoder** and generates a new sequence



RNN-based approach: problem

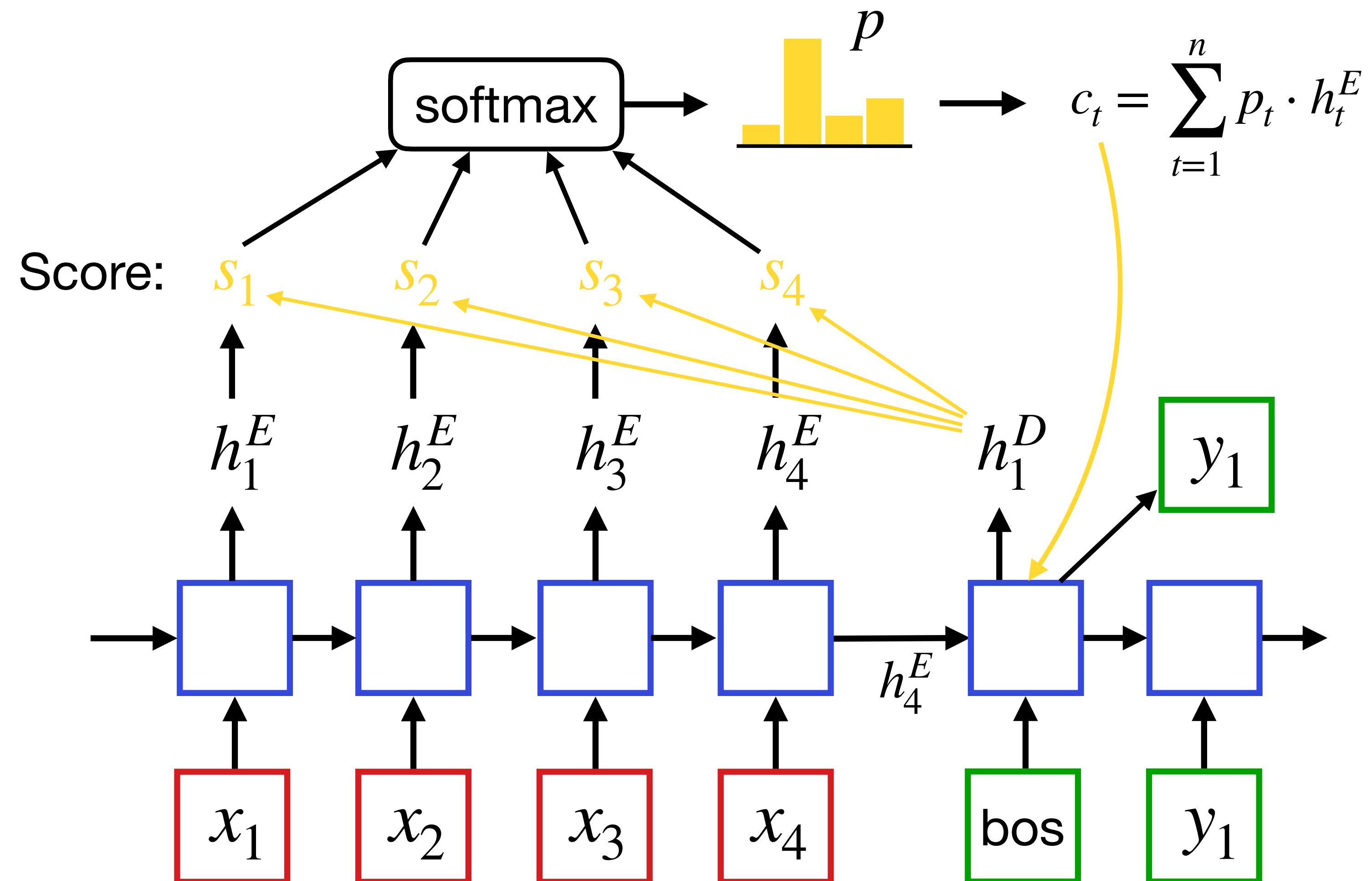
- The input text can be of any length
- We're trying to fit all the information about it into a single vector h
- We get a bottleneck



Attention mechanism

Idea:

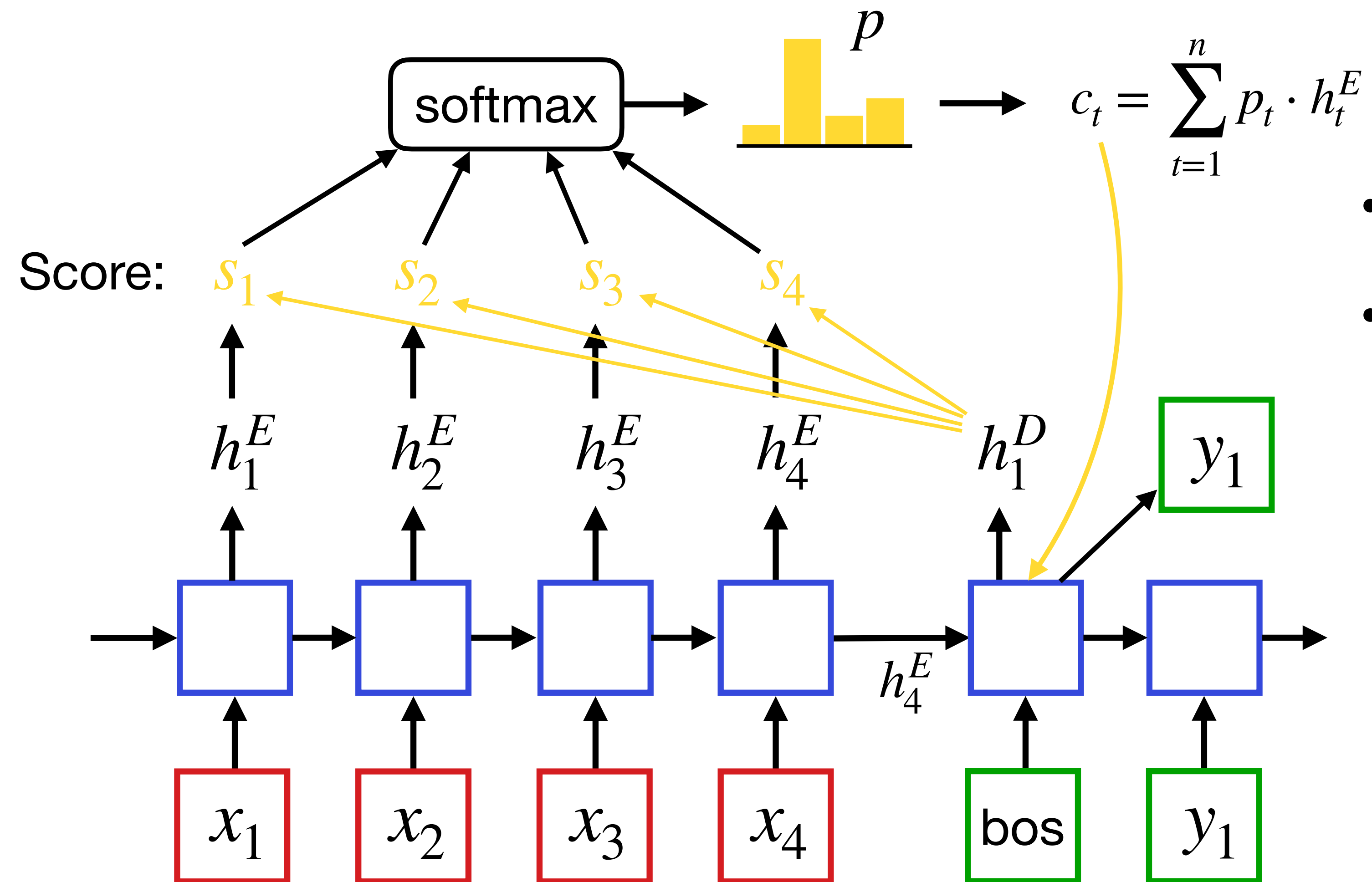
Allow the decoder to look at arbitrary encoder outputs



Attention mechanism

Idea:

Allow the decoder to look at arbitrary encoder outputs



- What is Score?
- How are vectors combined?

Score calculation methods

$$s(h_E, h_D) \in \mathbb{R}$$

1. Scalar product: $s(h_E, h_D) = \langle h_E, h_D \rangle$

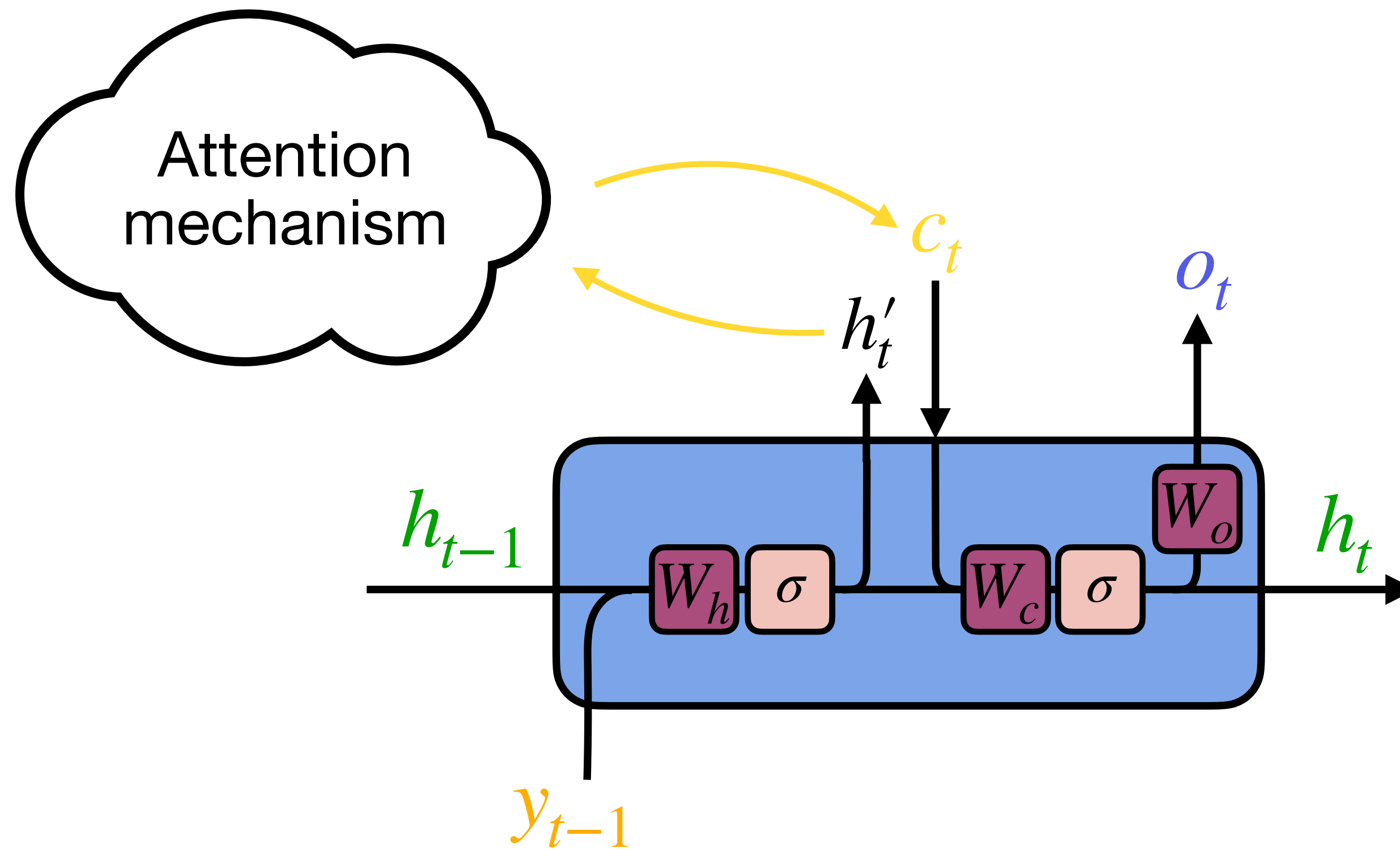
2. Bilinear form: $s(h_E, h_D) = h_E^T W h_D$

Learned matrix



3. Any arbitrary function

Decoder block



$$h'_t = \tanh(W_h[h_{t-1}, y_{t-1}] + b_h)$$

$$h_t = \tanh(W_c[h'_t, c_t] + b_c)$$

$$o_t = W_o h_t + b_o$$

Other methods:

[Bahdanau et al, 2014]

[Luong et al, 2015]

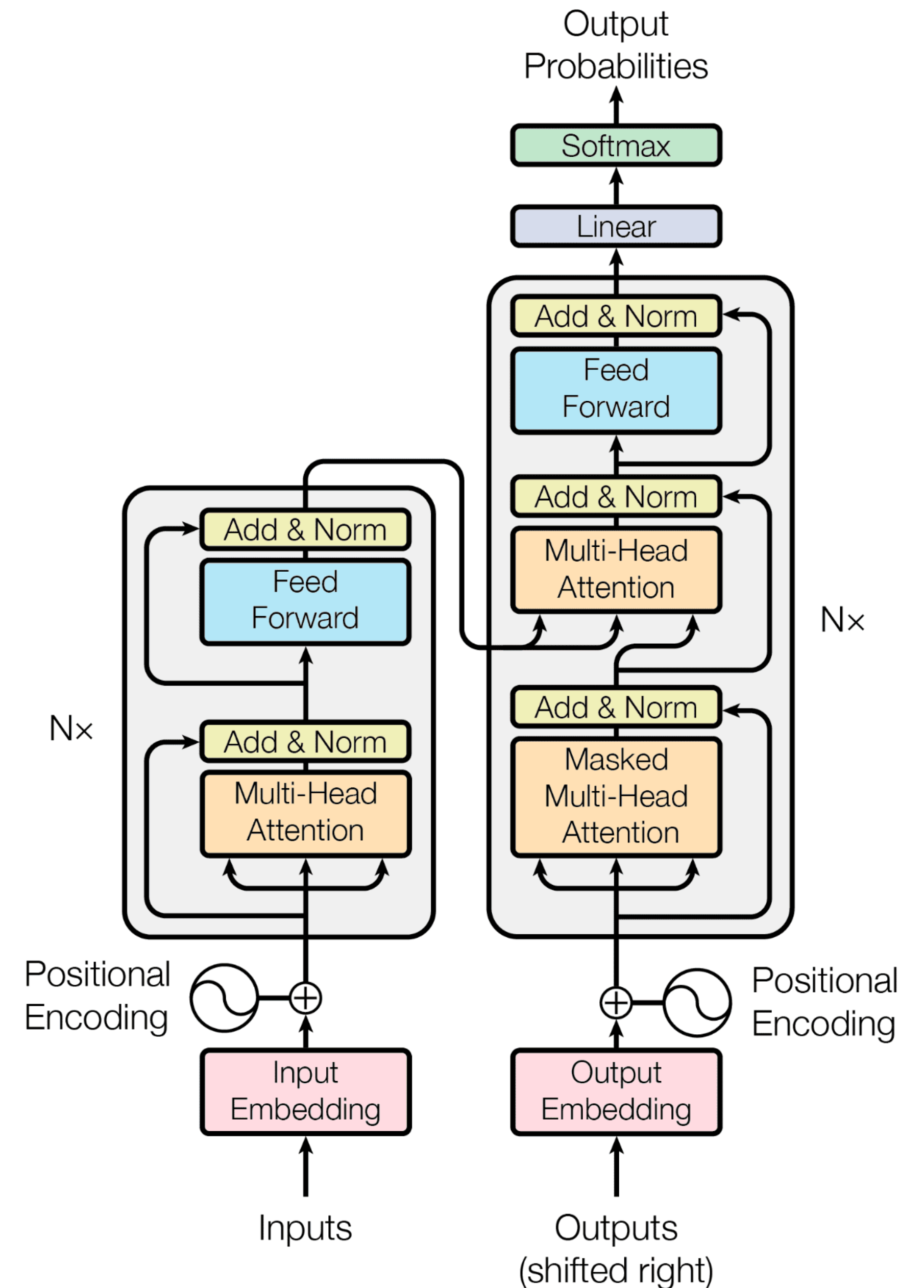
Transformer

Attention is all you need

- The most popular architecture since 2017
- ~200k citations for the original article

Components:

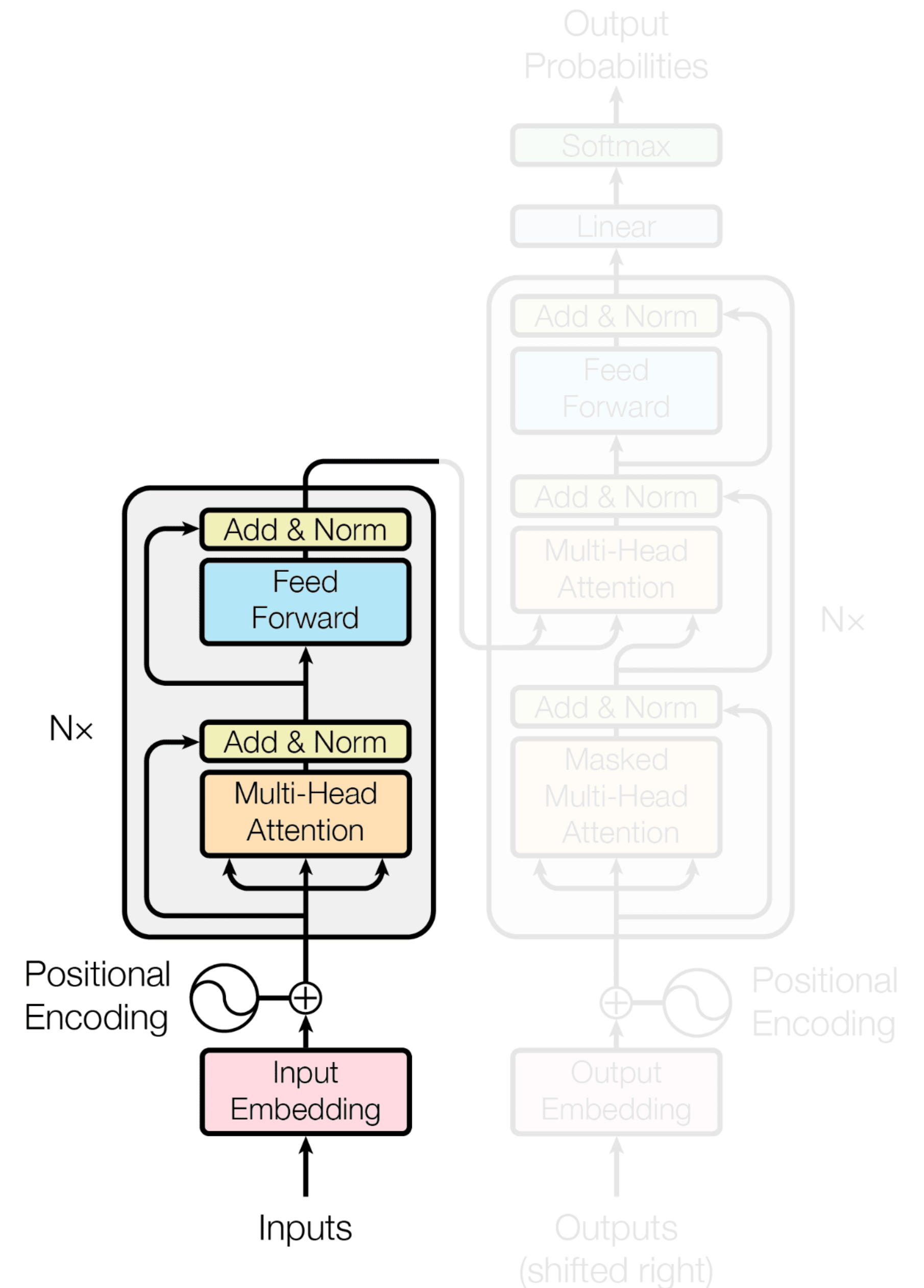
1. Multi-Head Self-attention
2. Feed Forward Network
3. Masked Multi-Head Self-attention
4. Positional Encodings



Encoder

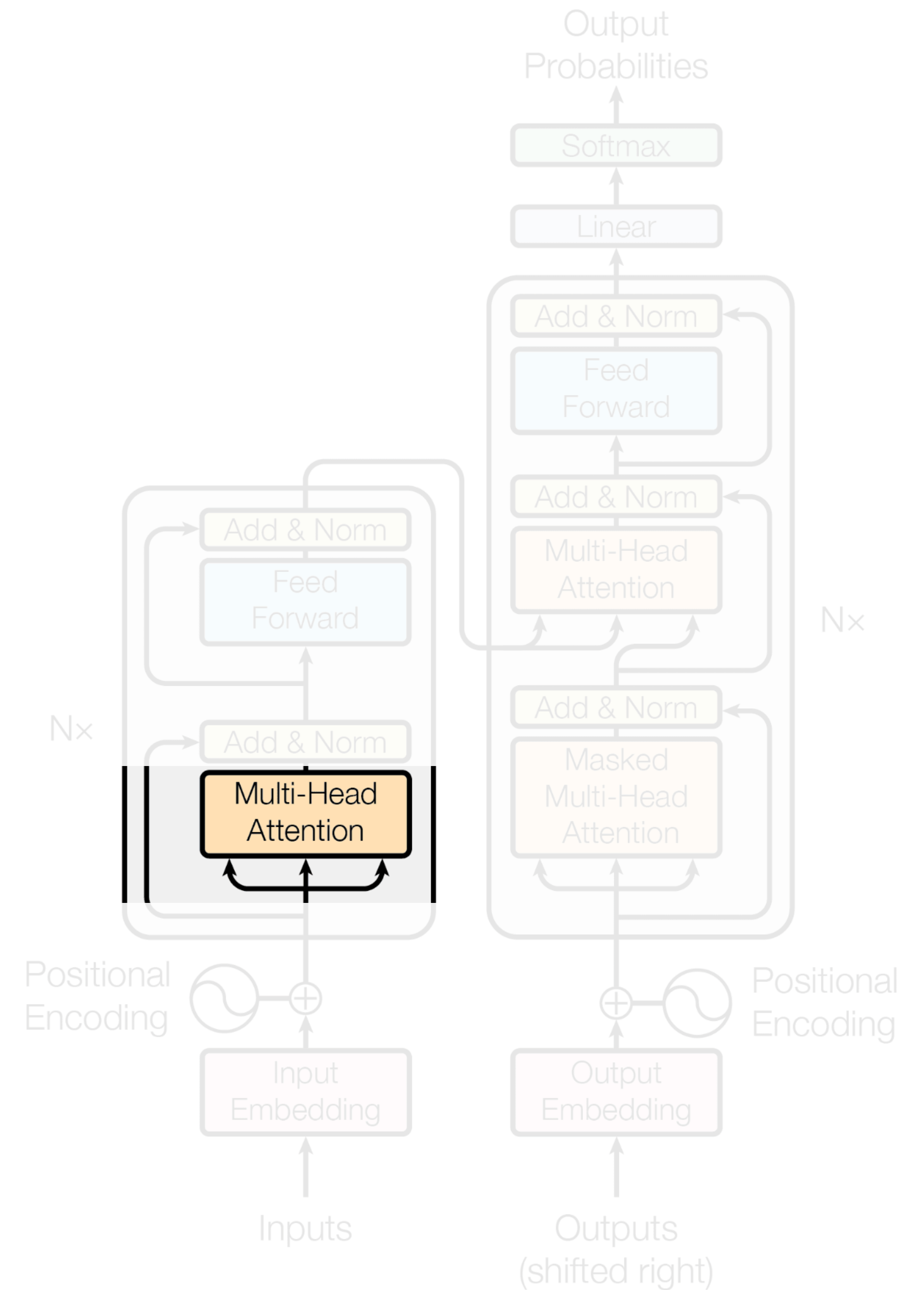
Encoder is used to extract features from a sequence

1. Multi-Head Self-attention
2. Positional Encoding
3. Feed Forward Network
4. Add & Norm



Encoder

1. **Multi-Head Self-attention**
2. Positional Encoding
3. Feed Forward Network
4. Add & Norm



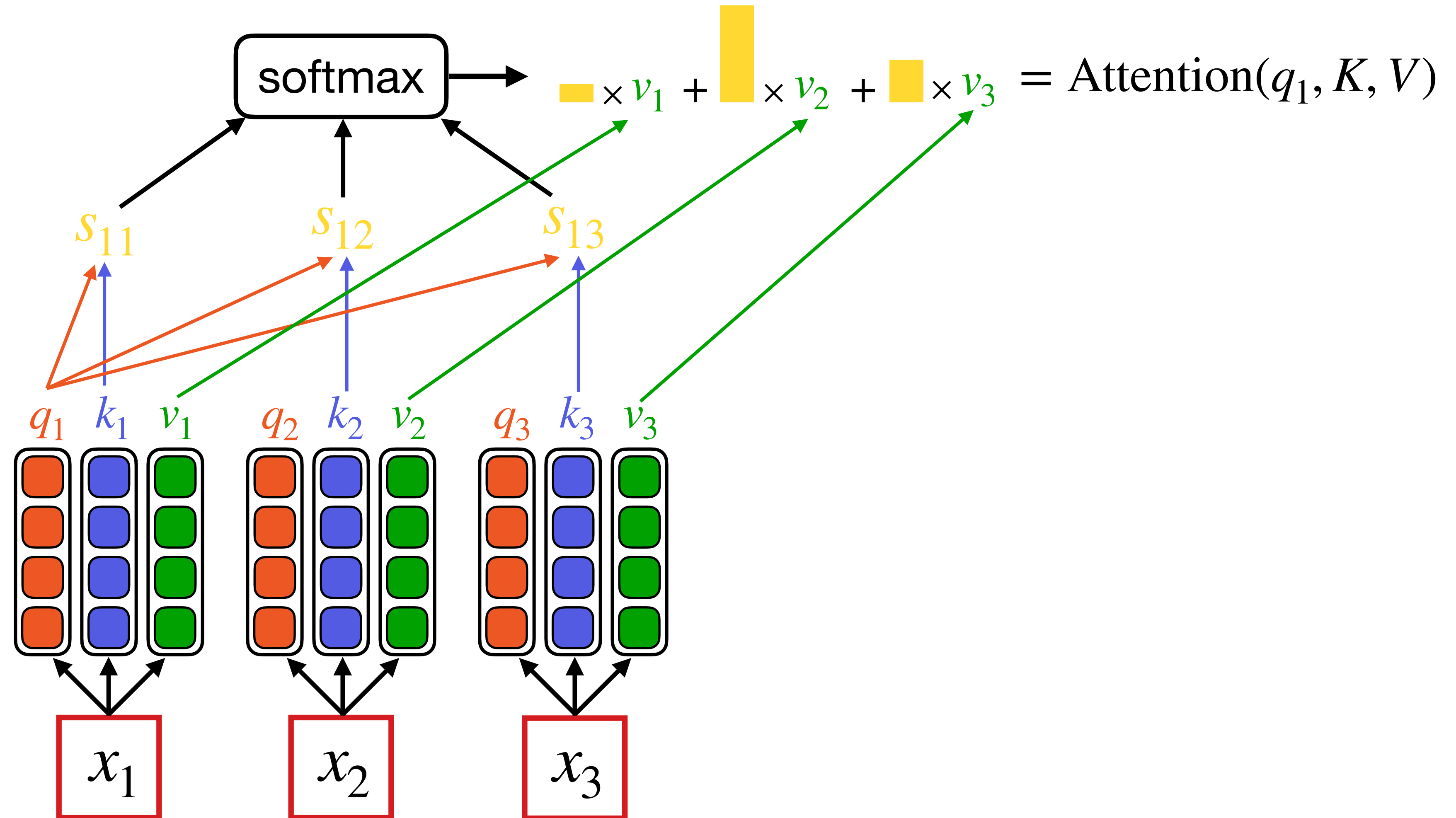
Self-attention

Allows each token to look at all the others

q – query

k – key

v – value



Self-attention

Allows each token to look at all the others

Query: $Q = xW_q + b_q$

Key: $K = xW_k + b_k$

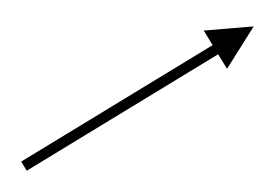
Value: $V = xW_v + b_v$

$$x \in \mathbb{R}^{n \times d}$$

$$W_q, W_k, W_v \in \mathbb{R}^{d \times d}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

normalization
constant



Multi-Head Self-attention

Self-attention only allows us to request information of one kind.
But what if we want to learn about different aspects?

A boy runs very fast

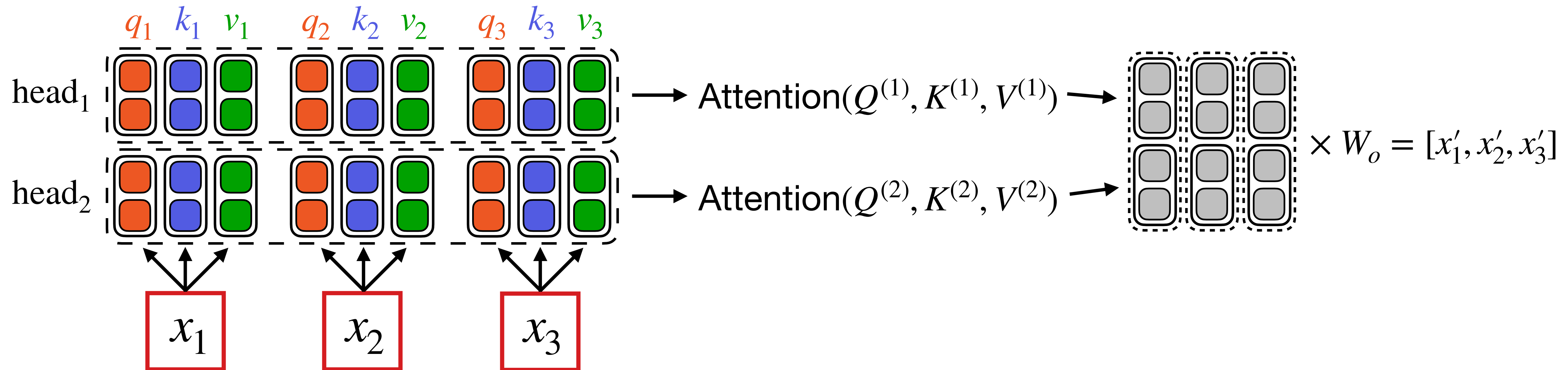


who? how?

Let's divide the attention into several heads

Multi-Head Self-attention

- Divide each vector q , k , v into equal parts
- Calculate self-attention for each part separately
- Concatenate the result
- Multiply by the output matrix



Multi-Head Self-attention

Query: $Q^{(i)} = xW_q^{(i)} + b_q^{(i)}$

Key: $K^{(i)} = xW_k^{(i)} + b_k^{(i)}$

Value: $V^{(i)} = xW_v^{(i)} + b_v^{(i)}$

$$\text{head}^{(i)} = \text{Attention}(Q^{(i)}, K^{(i)}, V^{(i)})$$

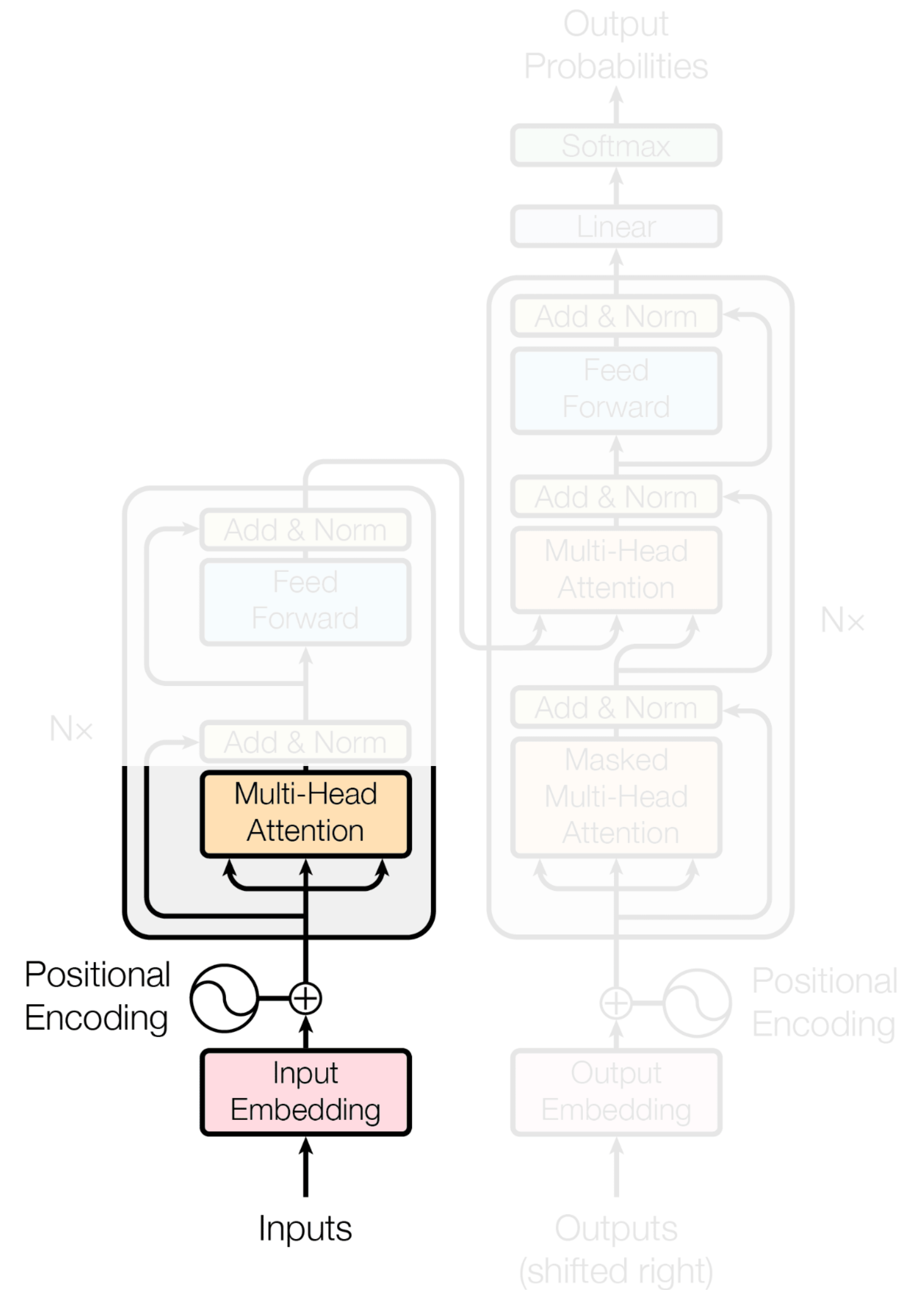
$$\text{MultiHead}(Q, K, V) = W_o \times [\text{head}^{(1)}, \dots, \text{head}^{(h)}]$$

$$W_q^{(i)}, W_k^{(i)}, W_v^{(i)} \in \mathbb{R}^{d_{\text{head}} \times d}$$

$$W_o \in \mathbb{R}^{d \times h \cdot d_{\text{head}}}$$

Encoder

1. Multi-Head Self-attention
2. **Positional Encoding**
3. Feed Forward Network
4. Add & Norm



Self-attention does not account for positions

Swapping tokens will not change the output!

$$w = \text{softmax} \left(\frac{q_i K^T}{\sqrt{d}} \right)$$

=> We cannot distinguish between the two cases

very good, **not** bad

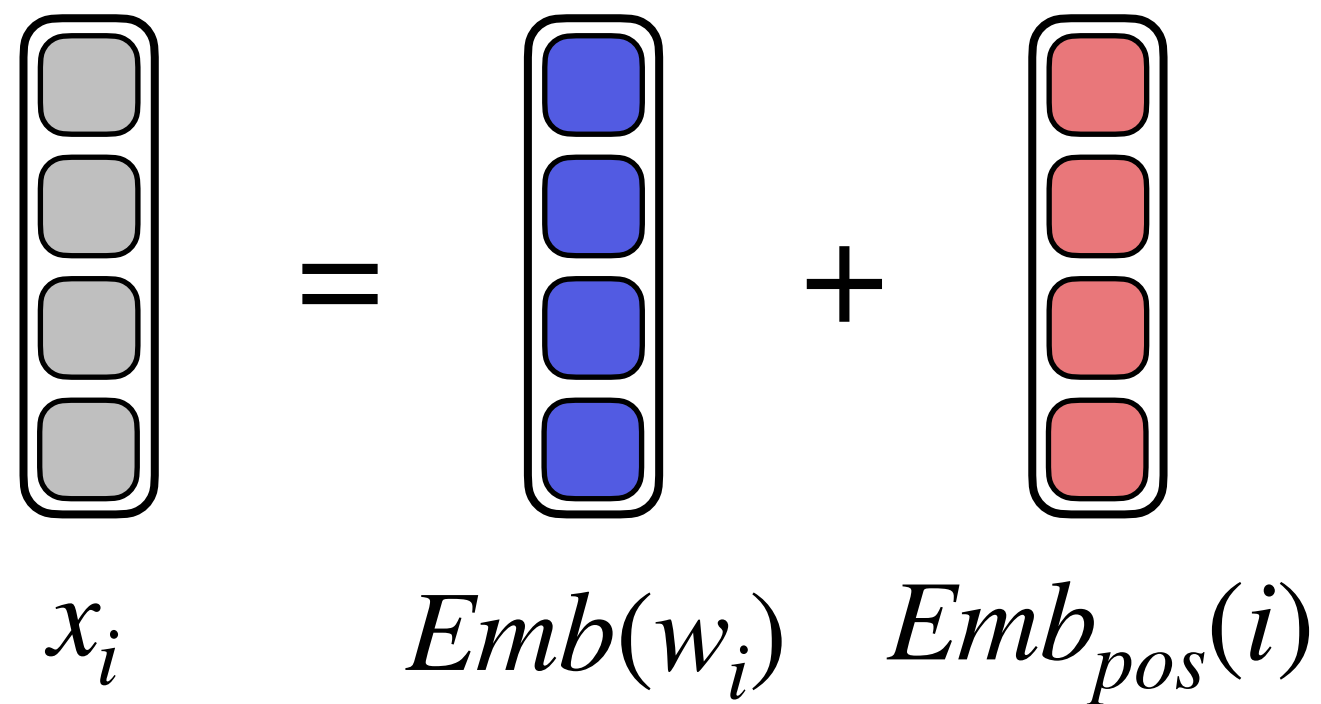
vs

very bad **not** good

$$\text{Attention}(q_i, K, V) = w_1 V_1 + \dots + w_n V_n$$

Positional Encoding

Necessary to account for positions of tokens in the text.



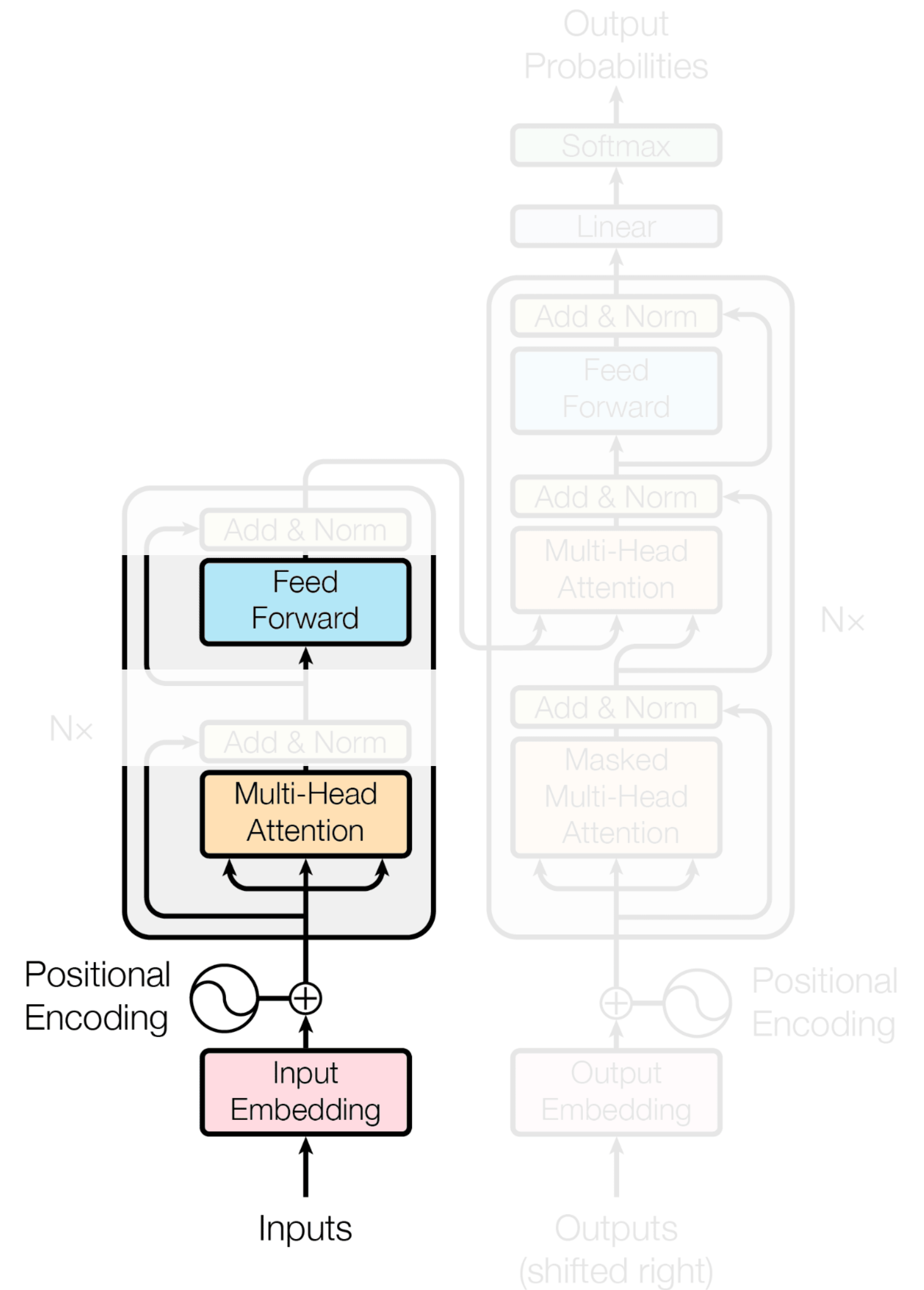
The diagram illustrates the formation of the input vector x_i . It consists of three vertical containers, each with four rounded rectangular slots. The first container, labeled x_i , has four gray slots. The second container, labeled $Emb(w_i)$, has four blue slots. The third container, labeled $Emb_{pos}(i)$, has four red slots. An equals sign is placed between the first and second containers, and a plus sign is placed between the second and third containers, indicating that the input vector x_i is the sum of the word embedding $Emb(w_i)$ and the position embedding $Emb_{pos}(i)$.

$$x_i = Emb(w_i) + Emb_{pos}(i)$$

Position embeddings are learned along with the rest of the model parameters.

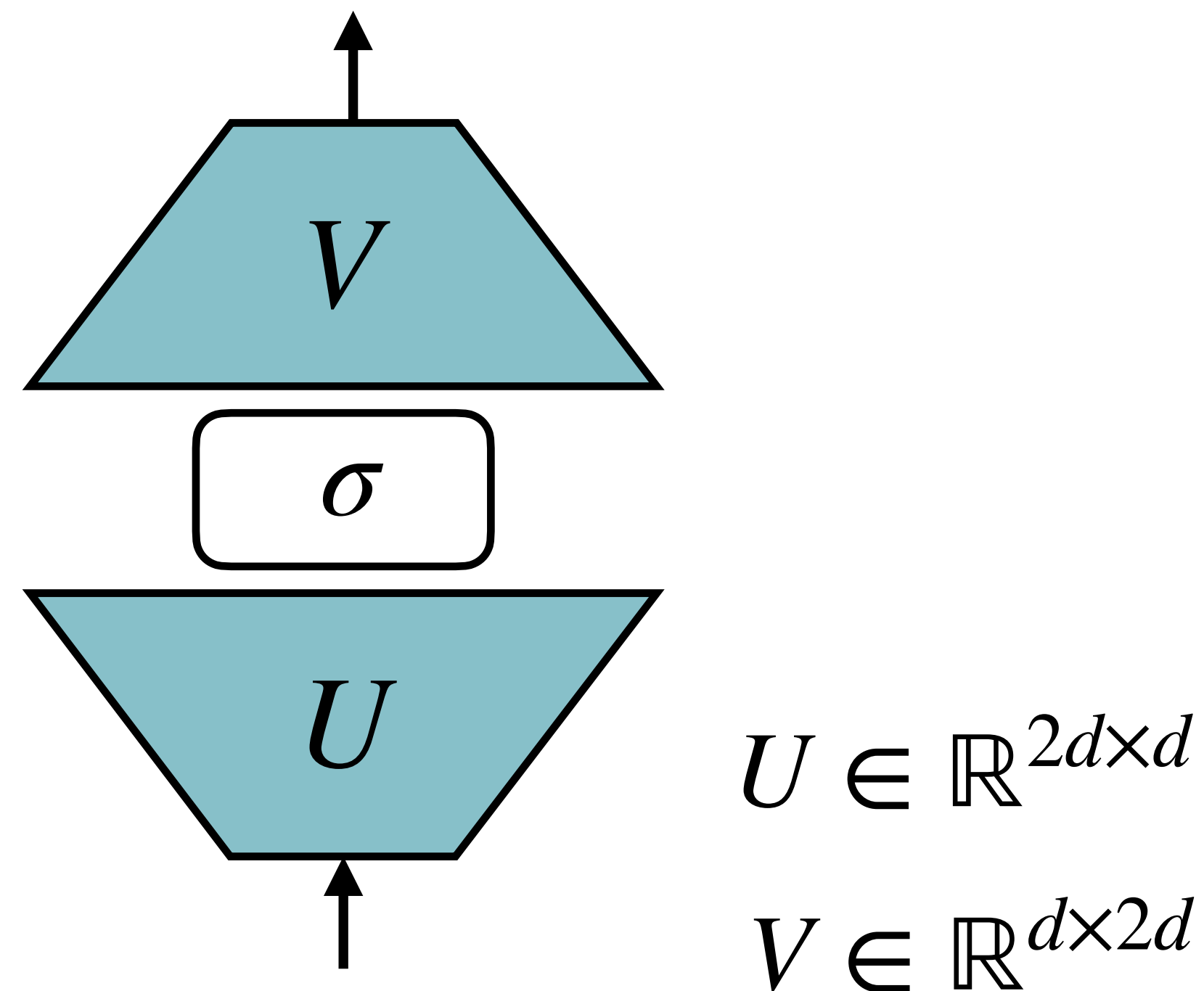
Encoder

1. Multi-Head Self-attention
2. Positional Encoding
3. **Feed Forward Network**
4. Add & Norm



Feed Forward Network

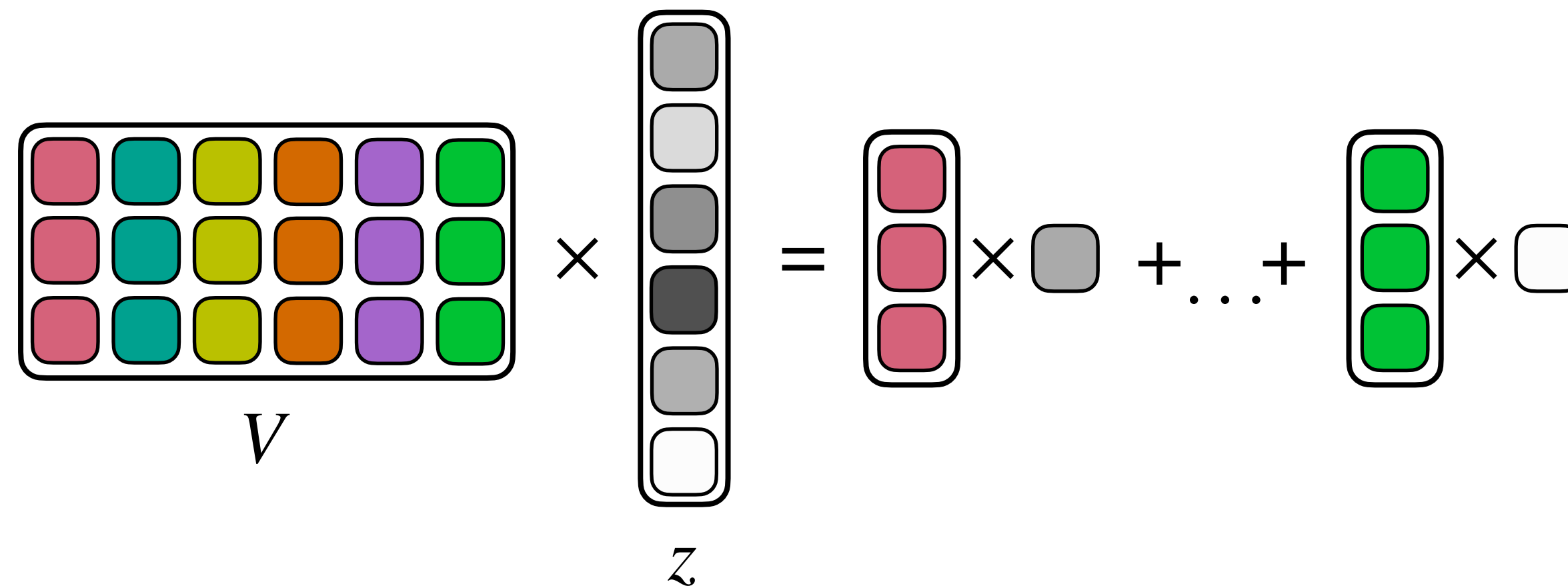
- A fully connected network with two layers.
- The first layer increases the dimensionality, the second layer reduces it.
- Used to process information obtained after self-attention.



FFN: Как ещё можно думать?

- FFN acts as a database!
- U contains keys, and V contains values.

$$\sigma(Ux_i) = z \in \mathbb{R}^{2d}$$

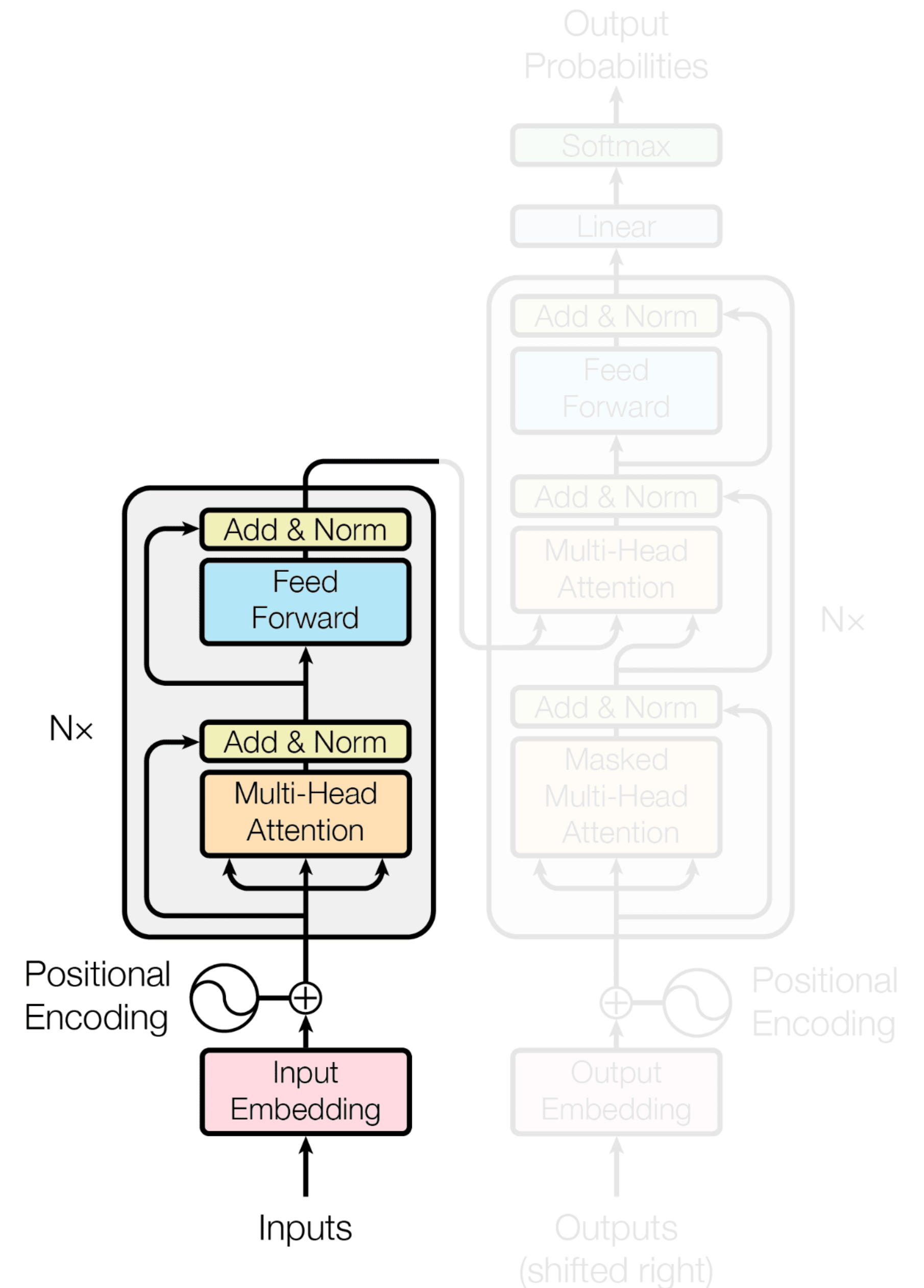


- Columns of V are specific facts
- We weight these facts according to the key z

Encoder

1. Multi-Head Self-attention
2. Positional Encoding
3. Feed Forward Network
4. **Add & Norm**

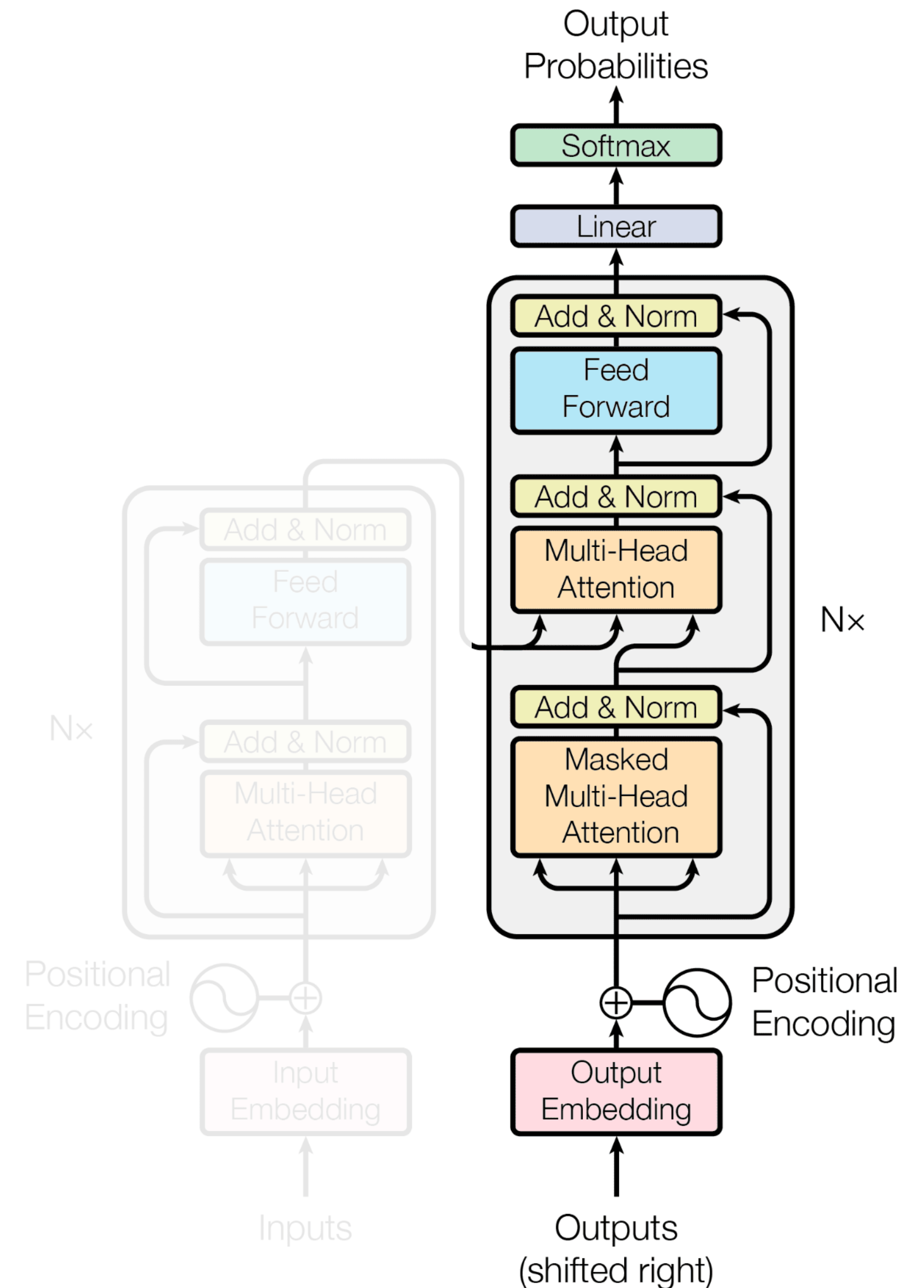
- Adding a **skip connection** to prevent gradient decay
- **Normalization** stabilizes the training



Decoder

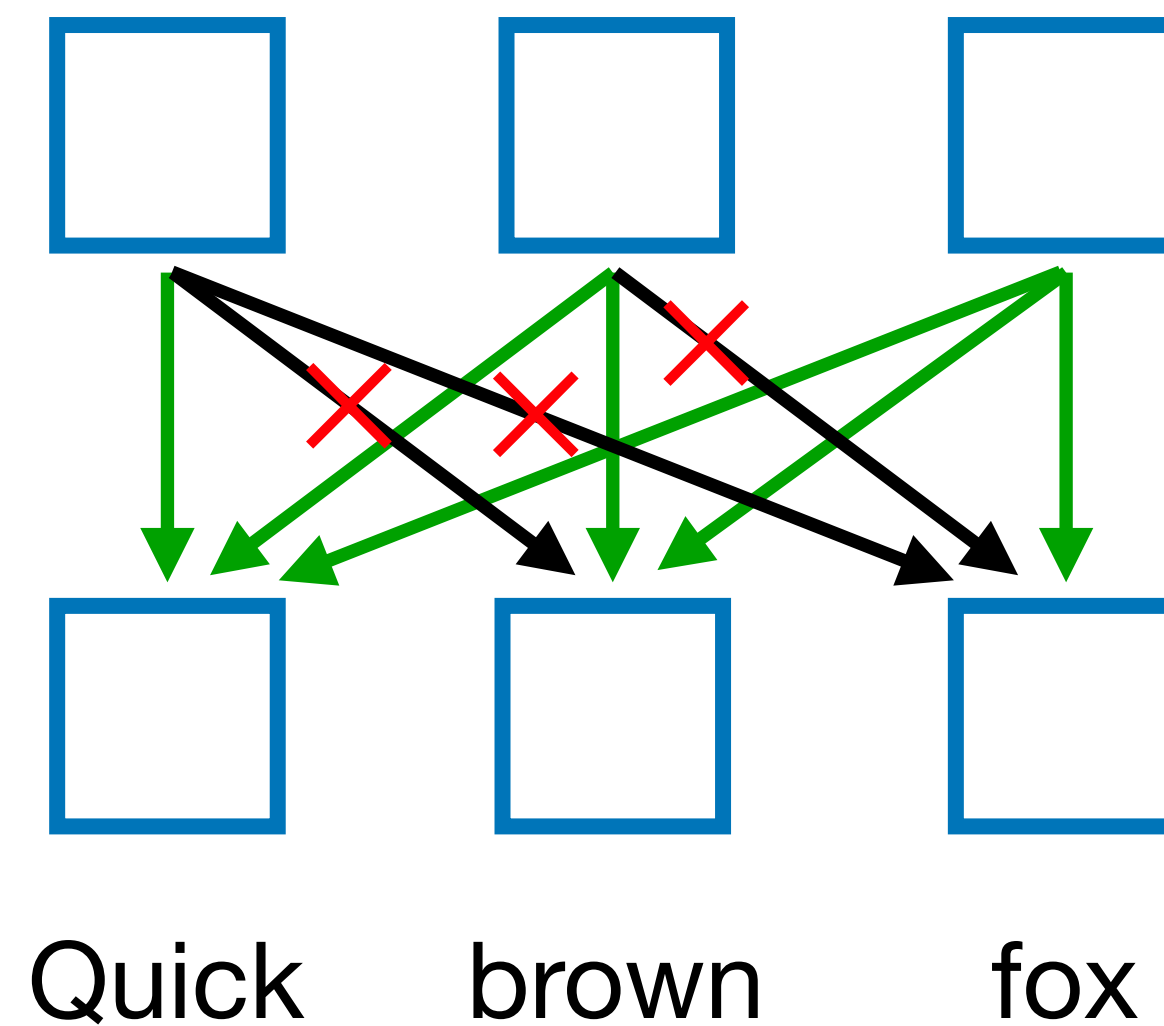
Decoder is used to generate the output text

1. Masked Multi-Head Self-attention
2. Cross Attention



Masked Self-attention

- When training the generator, we must prevent the decoder from looking at tokens that come after the current one.
- The decoder only considers other tokens in the self-attention layer.
=> It needs to be modified.



Masked Self-attention

- Adding a mask to the attention calculation
- The mask clearly prohibits looking forward

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + M\right) V$$

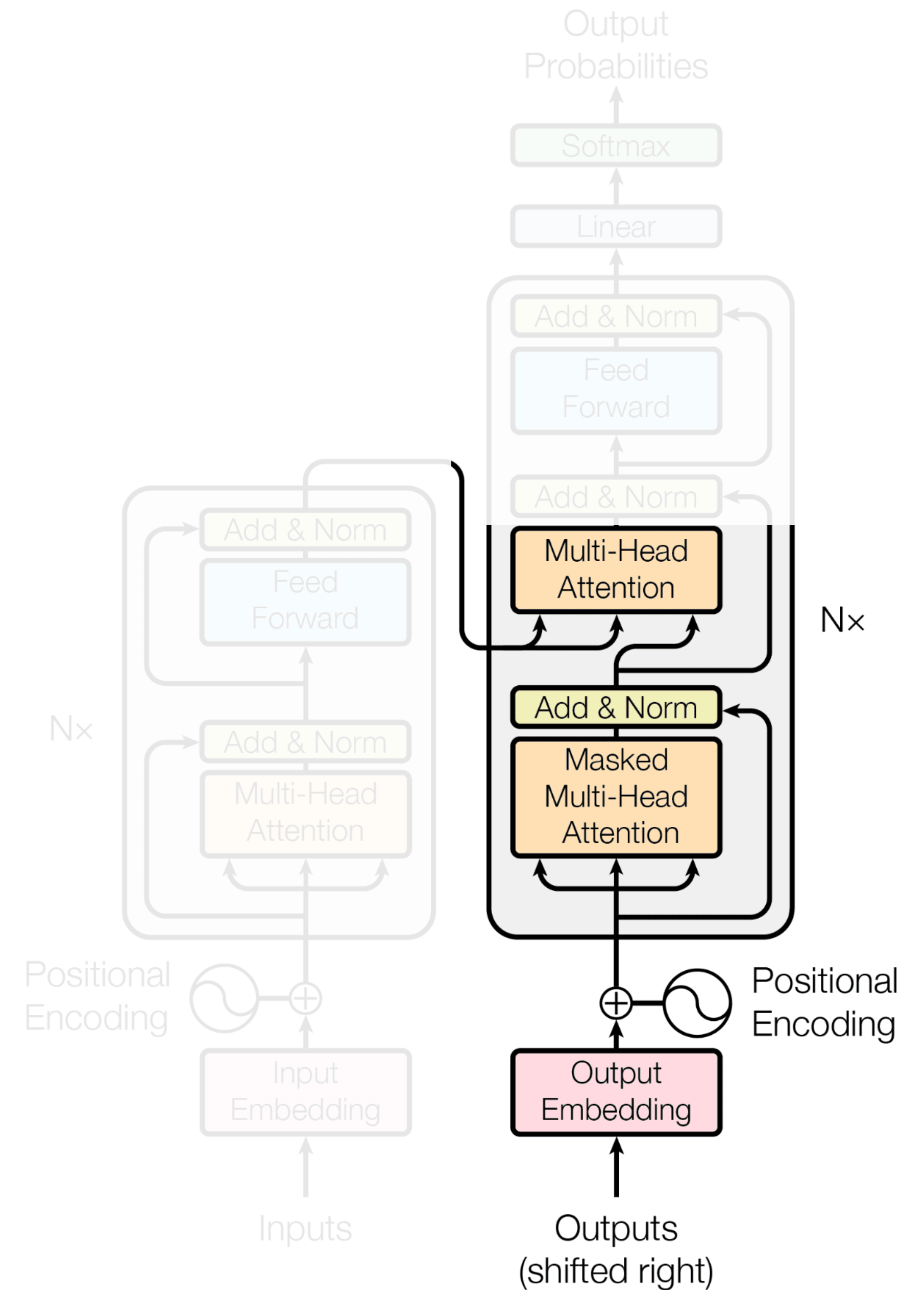
$$M_{ij} = \begin{cases} 0, & i \leq j \\ -\infty, & i > j \end{cases}$$

$M =$

0	$-\infty$	$-\infty$	$-\infty$
0	0	$-\infty$	$-\infty$
0	0	0	$-\infty$
0	0	0	0

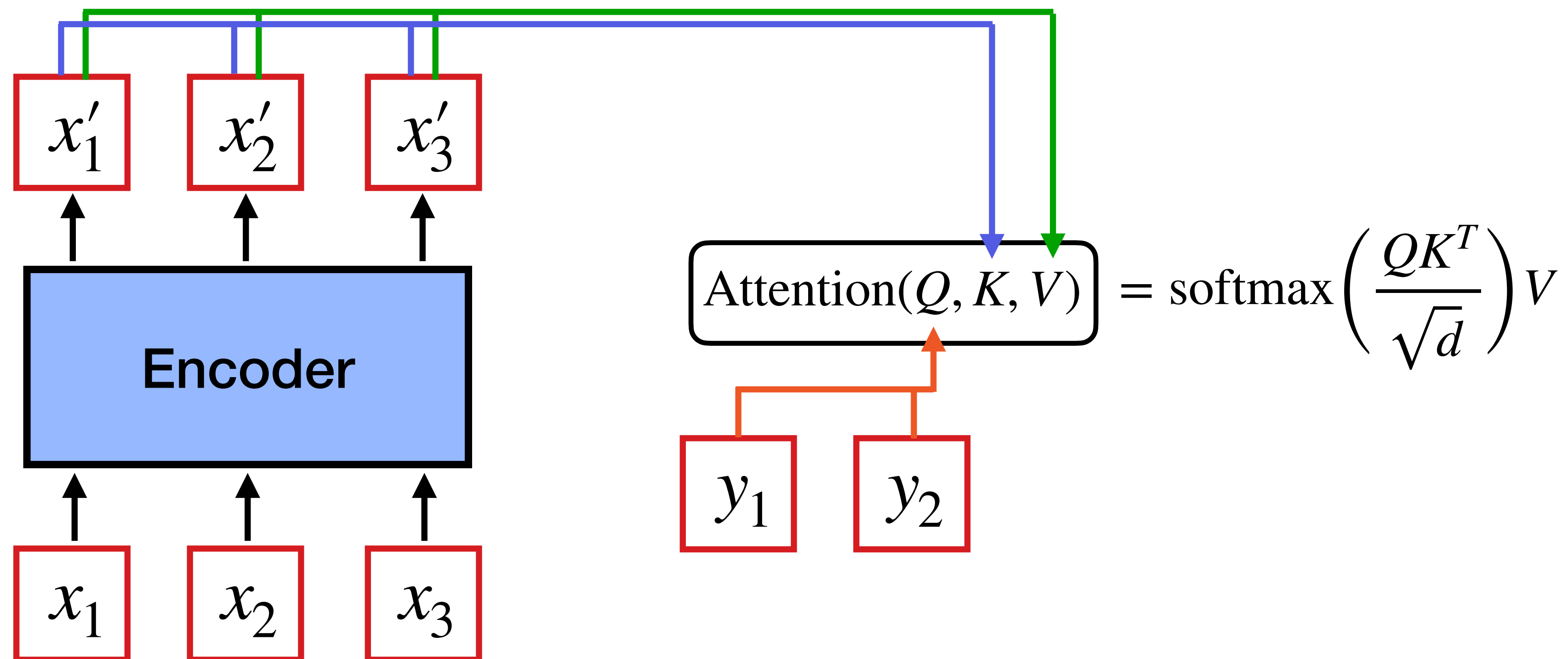
Decoder

1. Masked Multi-Head Self-attention
2. **Cross Attention**



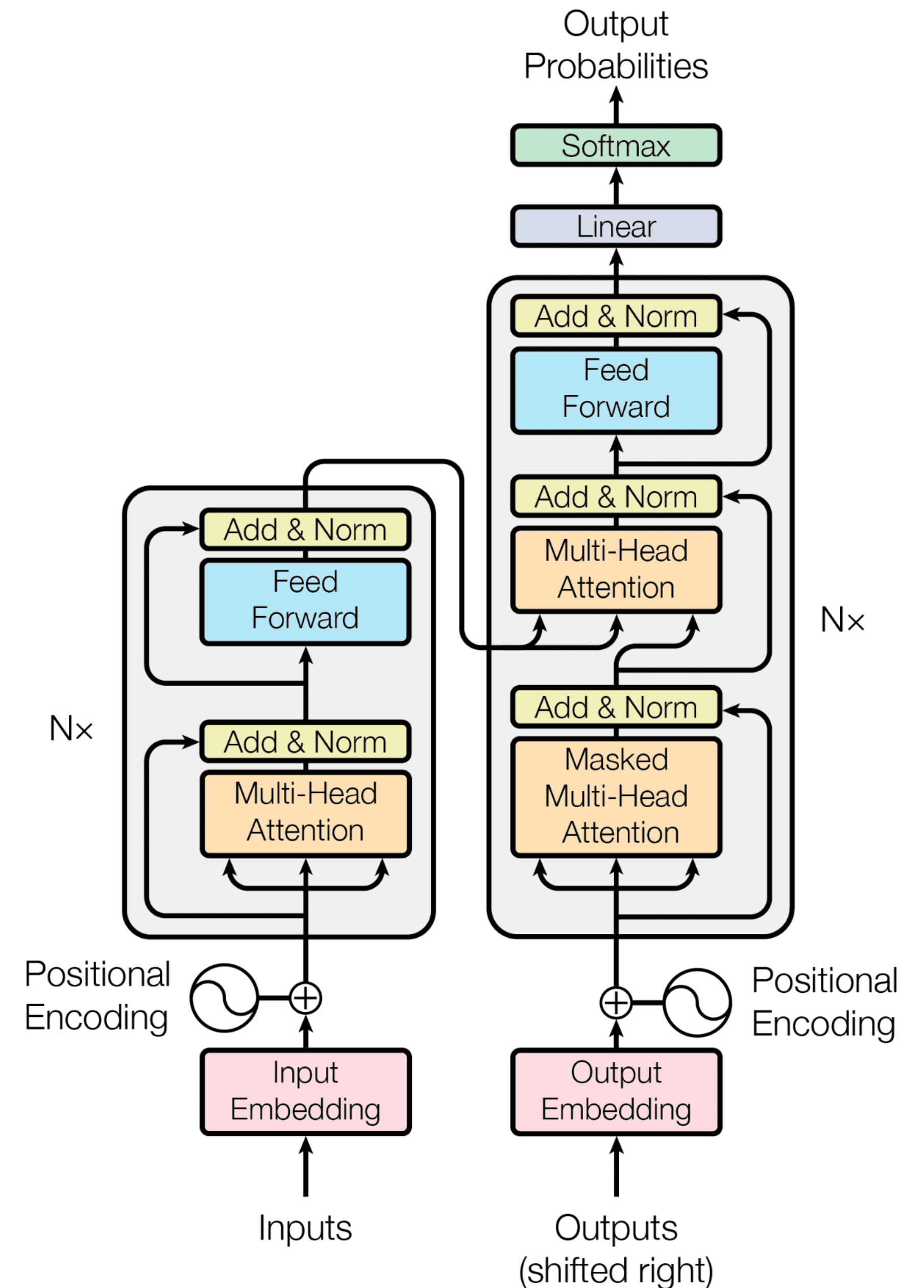
Cross Attention

- Used to pass information from the Encoder to the Decoder
- The Decoder "requests" information by passing query to attention
- The Encoder passes key and value



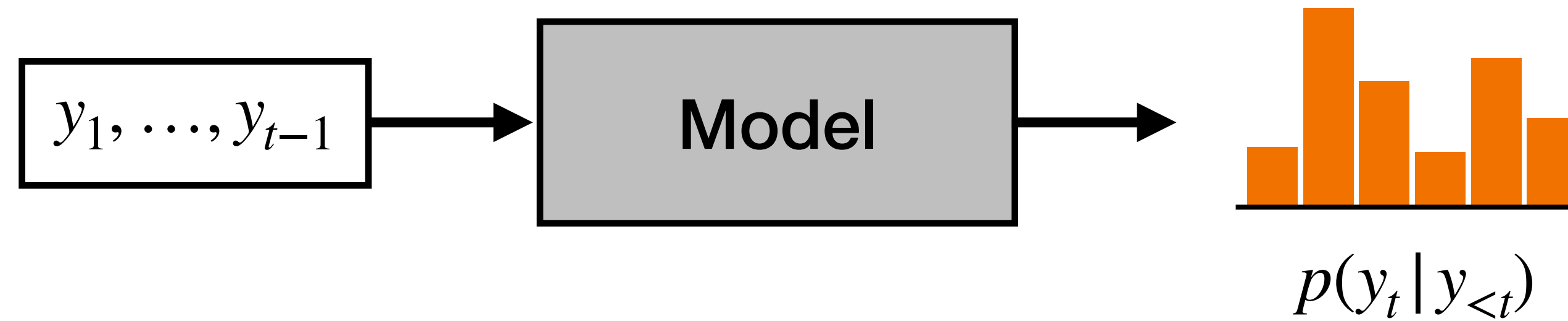
Summary

- The Transformer is designed specifically for Seq2seq tasks.
- The **Encoder** block consists of:
 - Multi-Head Attention
 - Feed Forward Network
 - After each layer, there is a skip connection and normalization
- The **Decoder** block uses
 - **Masked** Multi-Head Attention to avoid looking into the future
 - **Cross-Attention** to take information from the Encoder
 - At the output, the Decoder predicts the next token



Token sampling methods

The model produces a probability distribution of tokens



How to choose a token from this distribution?

Greedy Sampling

We can always choose the token with the highest probability

$$y_t = \arg \max_{y \in V} p(y | y_{<t})$$

Pros:

- Maximizes text likelihood

Cons:

- Loss of text diversity
 - It's bad if the generation is **unconditional**
 - It's not so bad if it's **conditional** (seq2seq)
- 

Beam Search

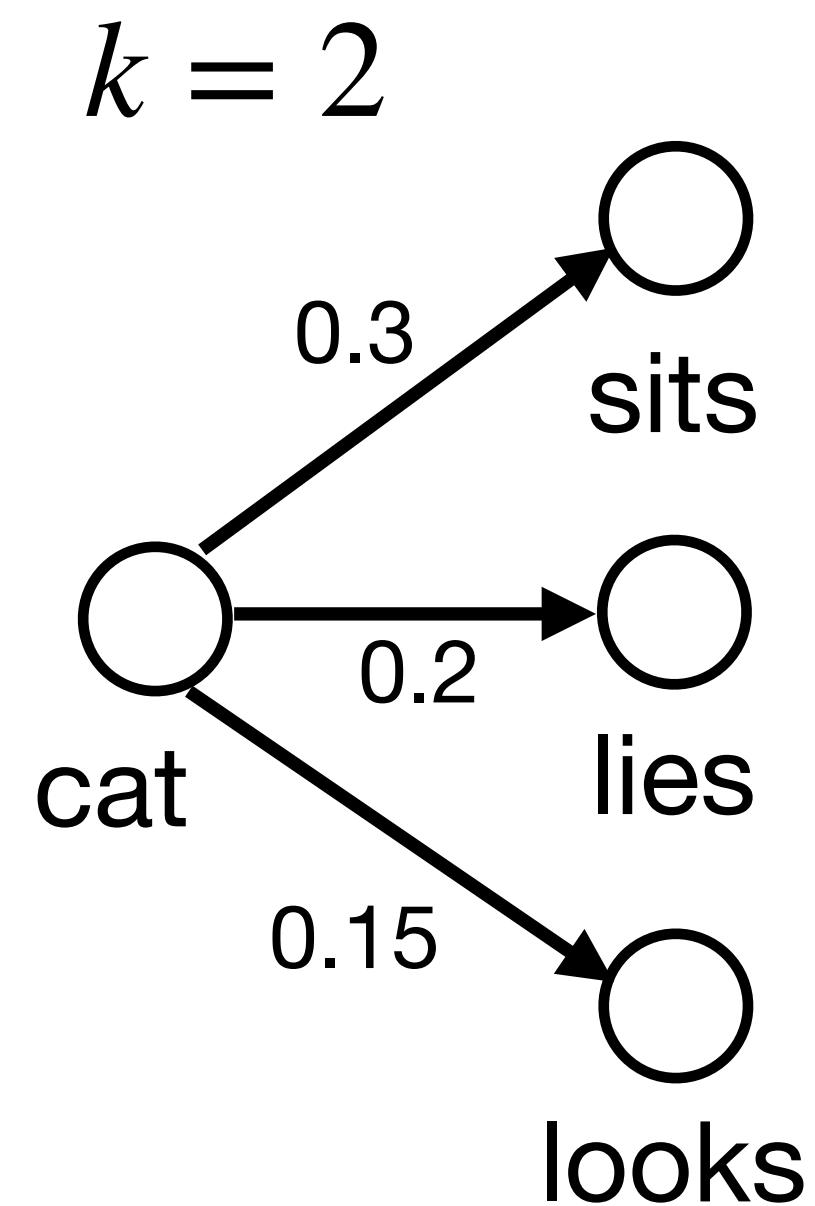
- Let's try to maximize the text's probability even further.
- With greedy sampling, we don't take into account the influence of a token on **subsequent** tokens.
- We'll make predictions several tokens ahead, and then choose the token with the highest cumulative probability.

Beam Search

Idea: Maintain the k most probable trajectories

At each step, generate k continuations for each trajectory

Keep only the k most probable ones

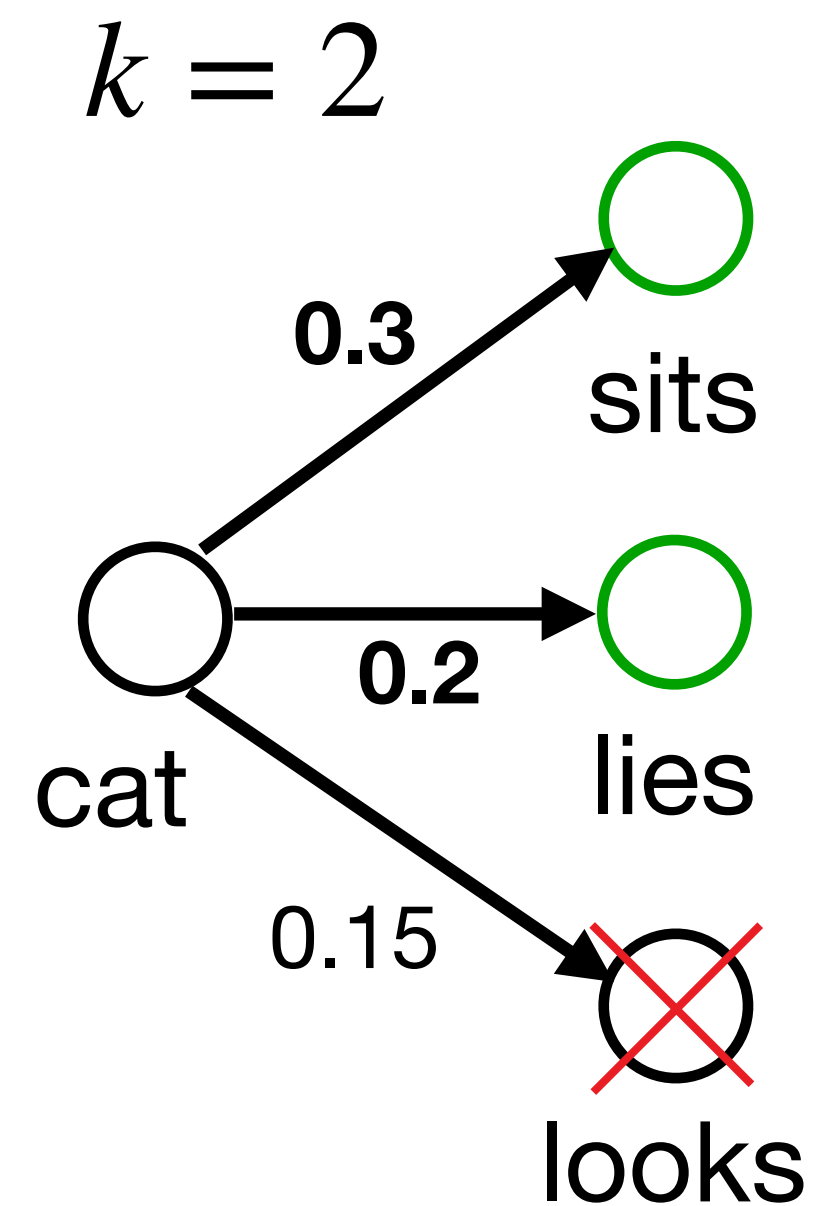


Beam Search

Idea: Maintain the k most probable trajectories

At each step, generate k continuations for each trajectory

Keep only the k most probable ones

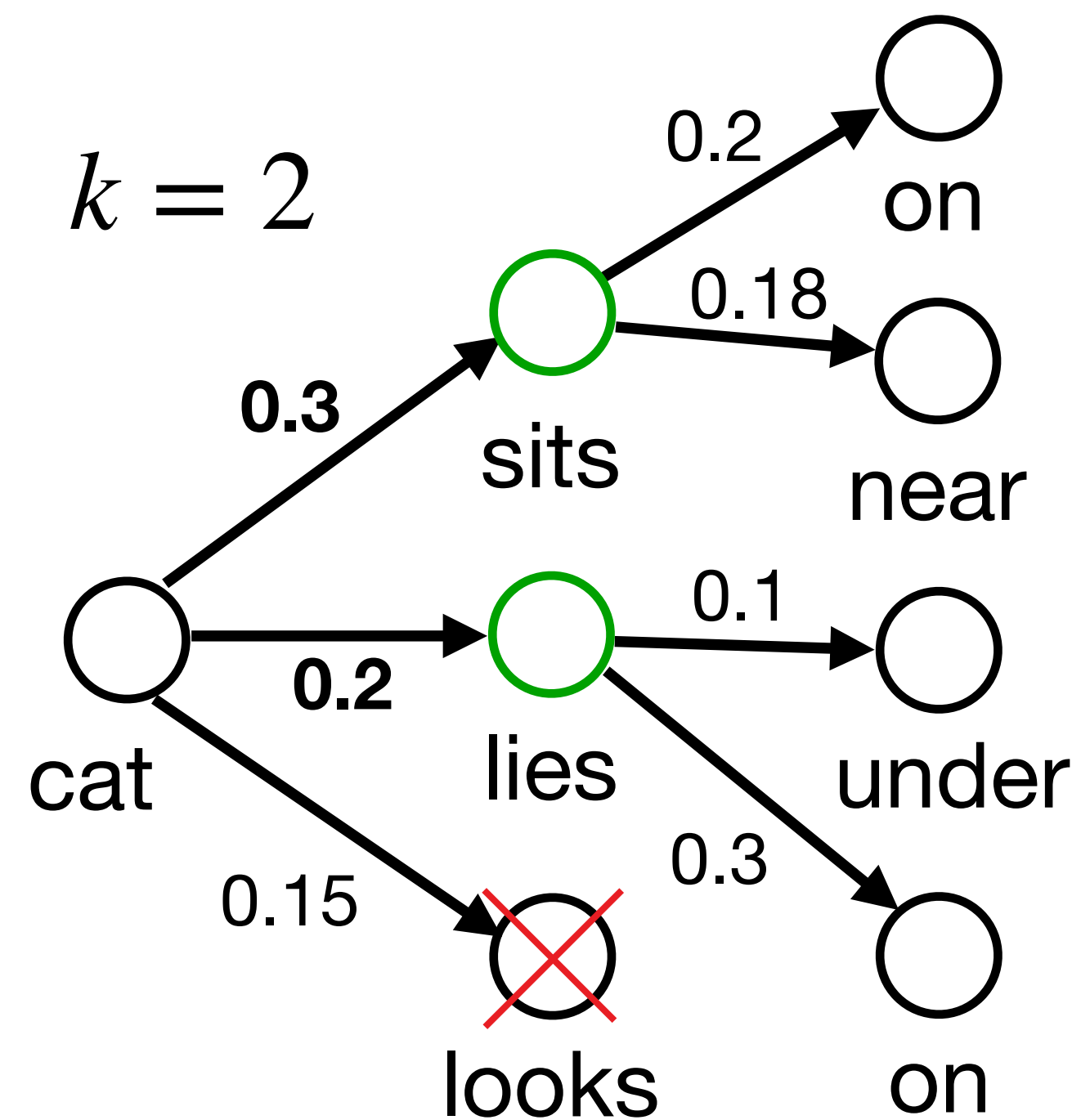


Beam Search

Idea: Maintain the k most probable trajectories

At each step, generate k continuations for each trajectory

Keep only the k most probable ones

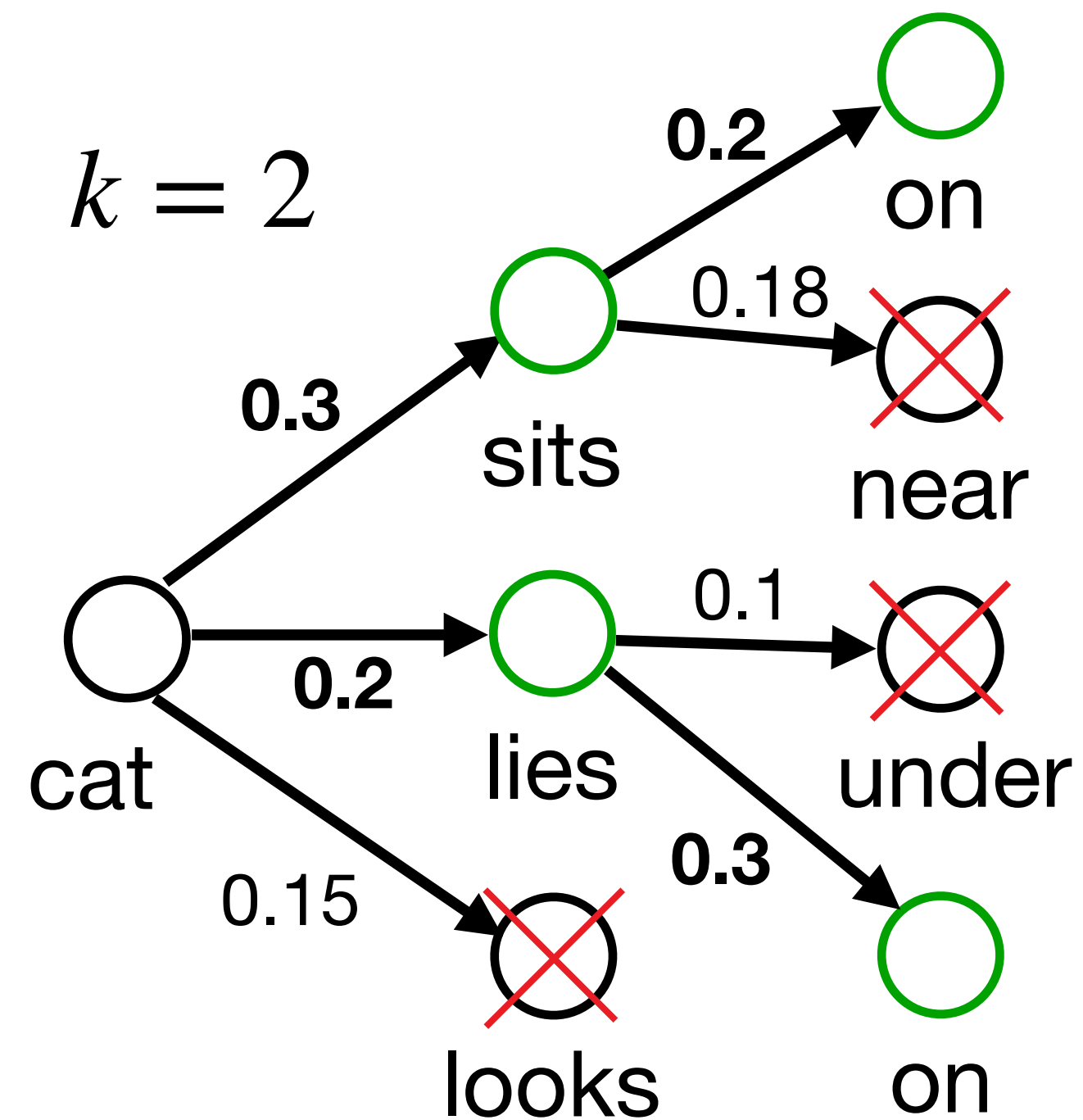


Beam Search

Idea: Maintain the k most probable trajectories

At each step, generate k continuations for each trajectory

Keep only the k most probable ones

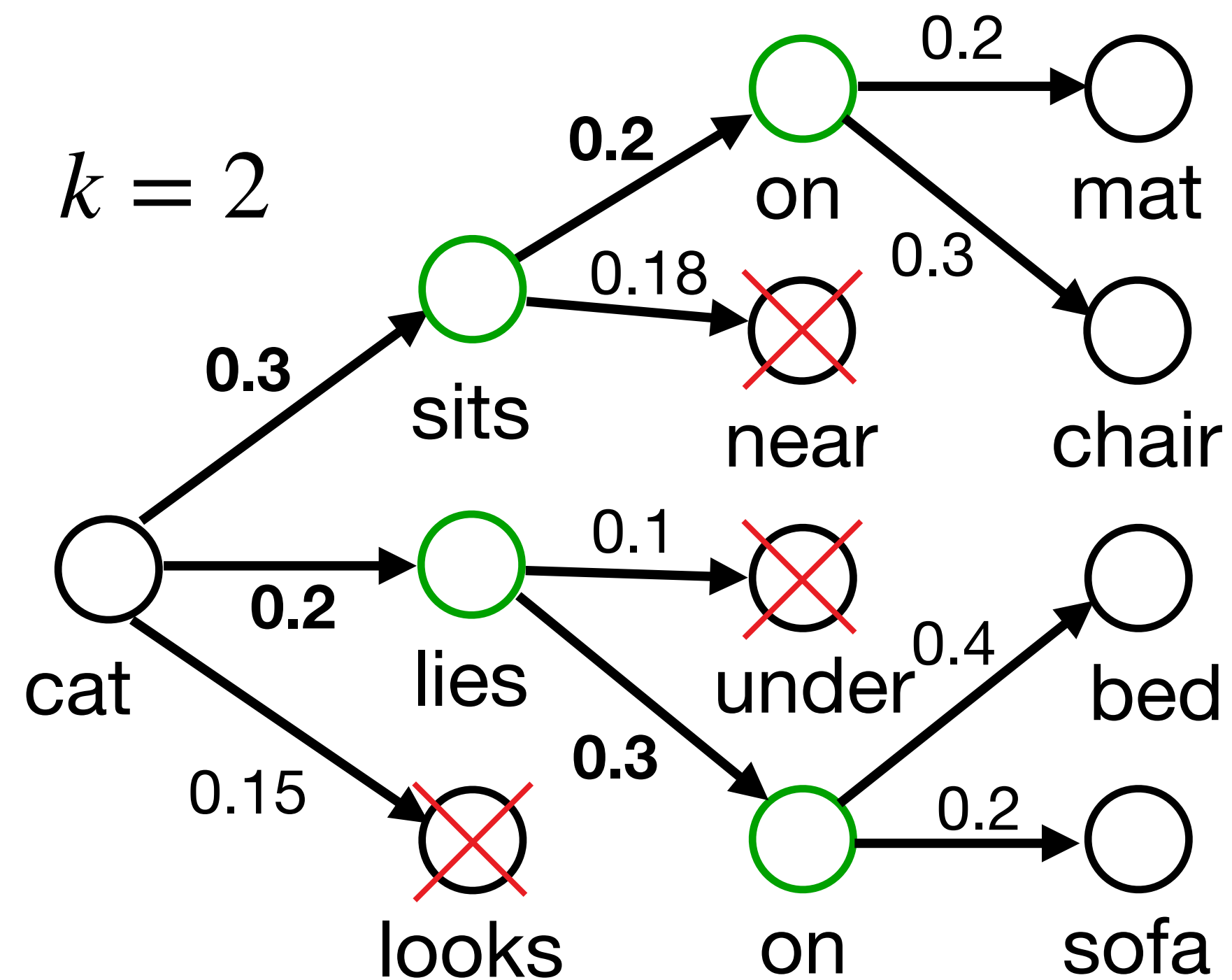


Beam Search

Idea: Maintain the k most probable trajectories

At each step, generate k continuations for each trajectory

Keep only the k most probable ones

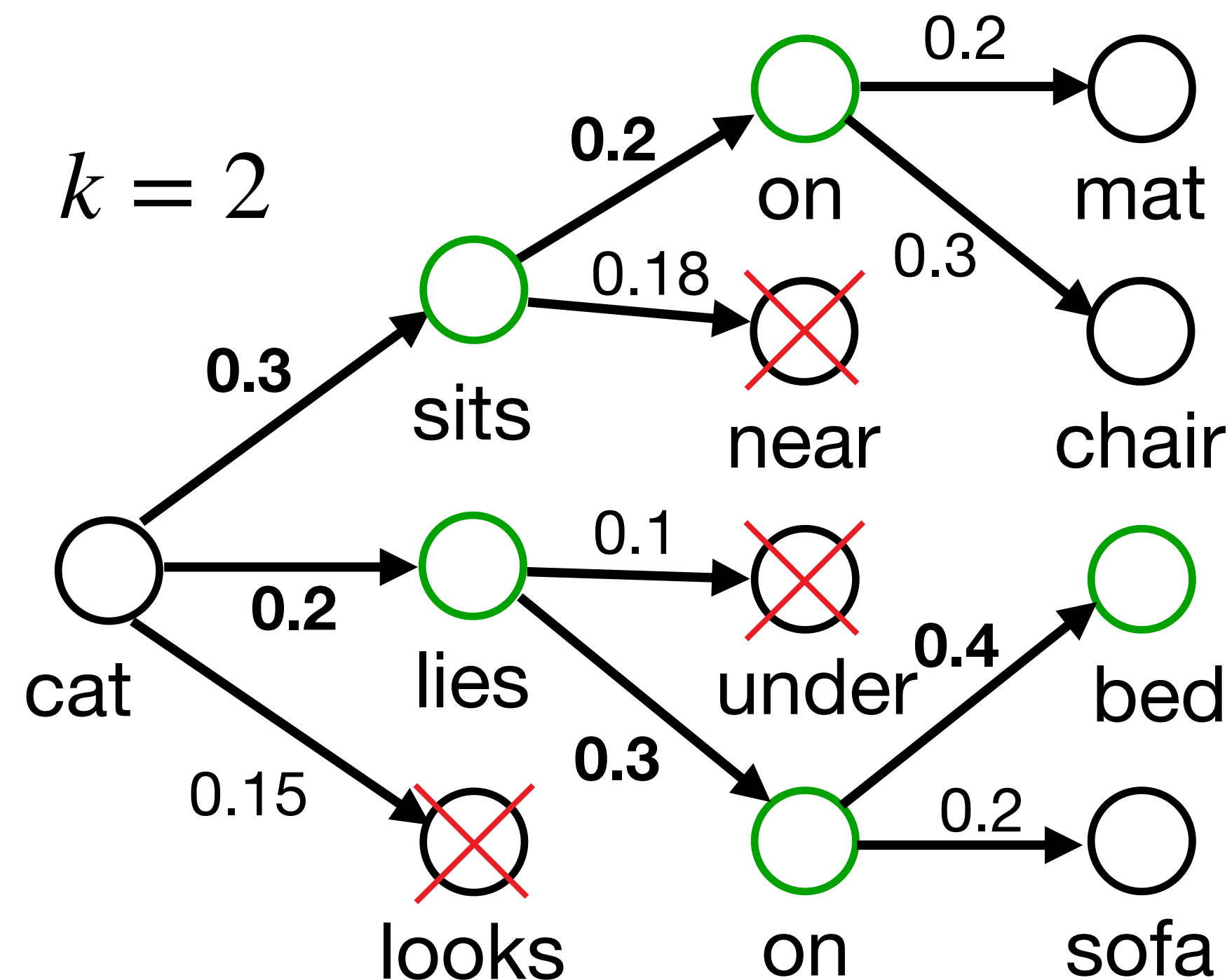


Beam Search

Idea: Maintain the k most probable trajectories

At each step, generate k continuations for each trajectory

Keep only the k most probable ones



The trajectory with "lies" has the highest probability

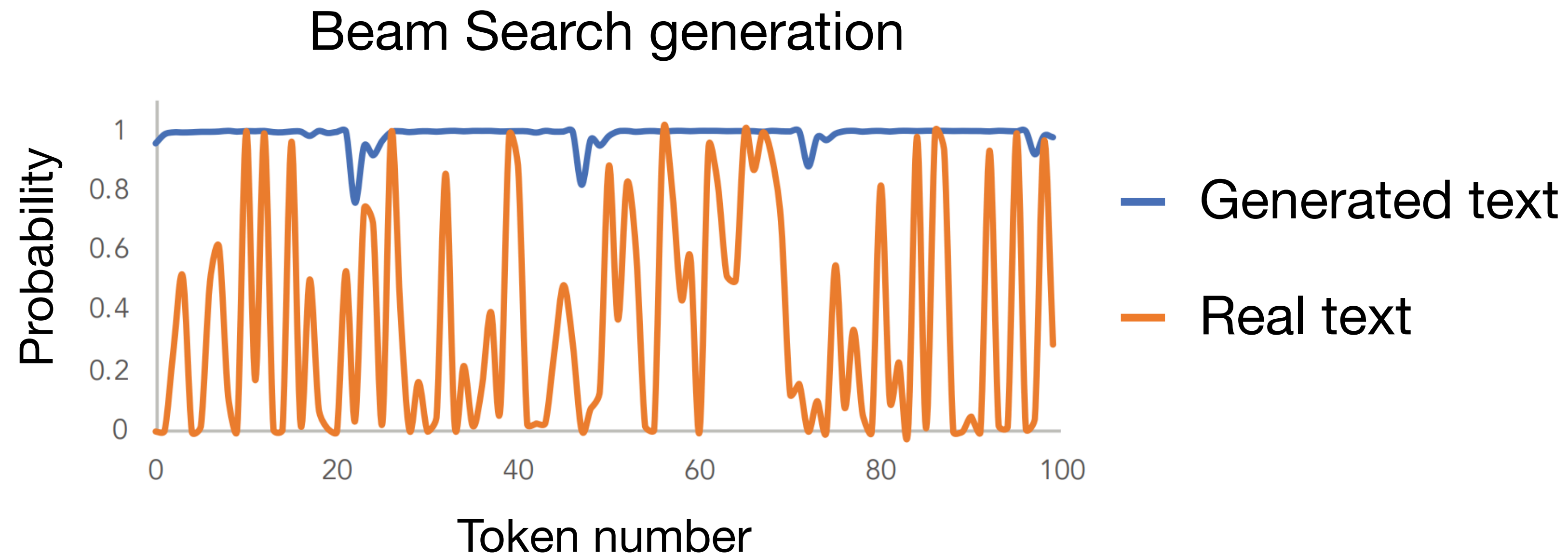
=> choose "lies"

Beam Search vs Greedy sampling

- Beam Search performs better for all seq2seq tasks
- Beam Search is much slower (depending on k)
- Popular values of k are 3 or 5

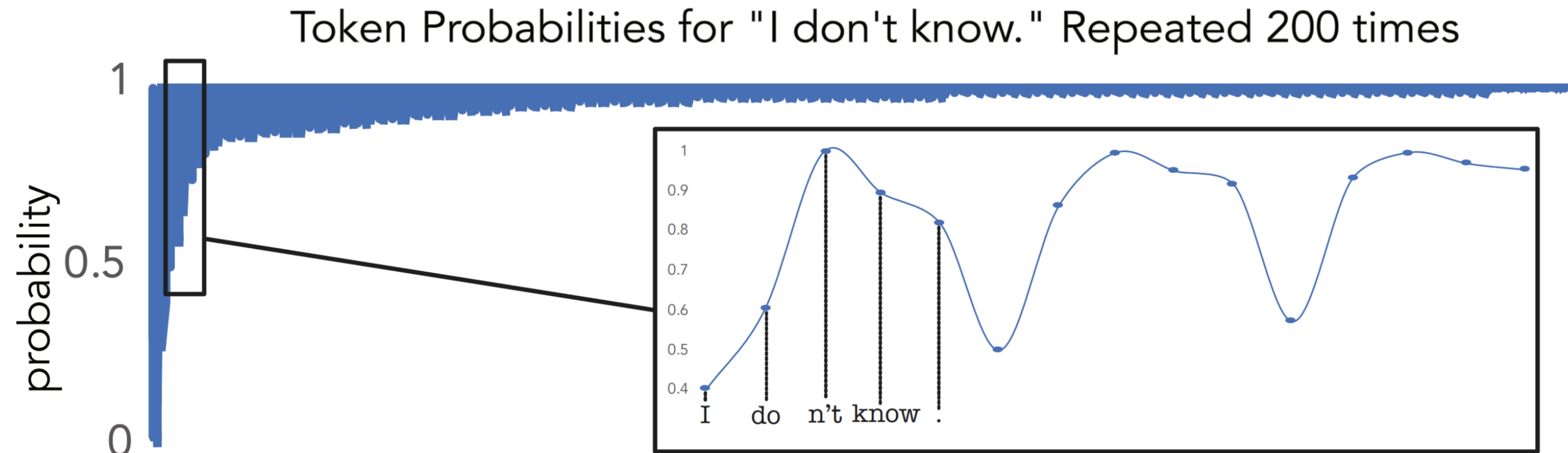
Unconditional generation

Both methods reduce the diversity of unconditional generation!



Unconditional generation

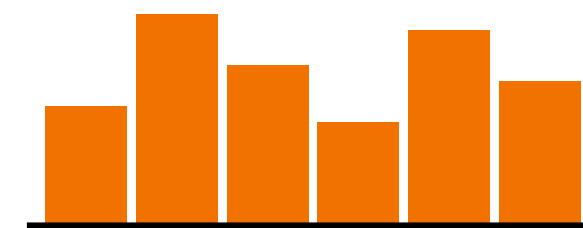
Both methods encourage repetition!



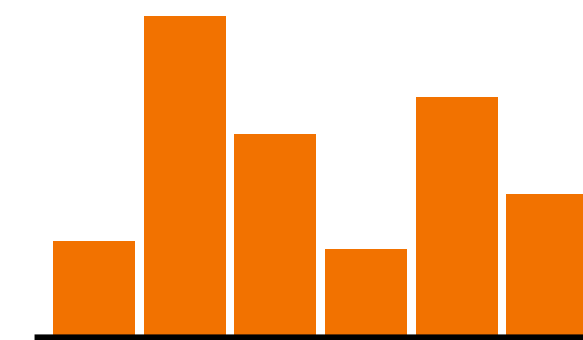
Sampling with temperature

- With conventional probability sampling, the probability mass of random tokens is too large.
- Let's make the distribution more degenerate by adding temperature $\tau \in [0,1]$

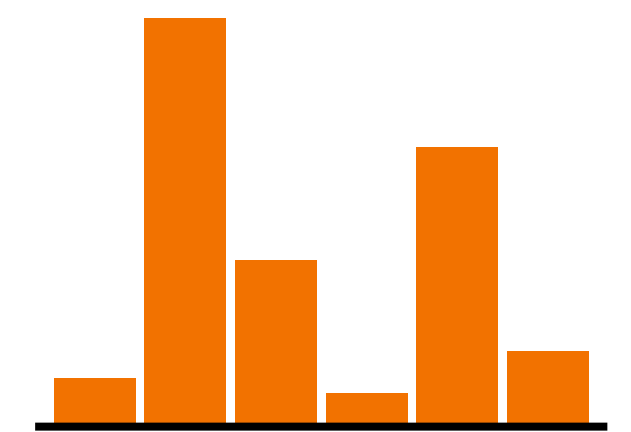
$$p_{\tau}(y | y_{<t}) = \frac{\exp \frac{p(y | y_{<t})}{\tau}}{\sum_{w \in V} \exp \frac{p(w | y_{<t})}{\tau}}$$



$p_2(y_t | y_{<t})$



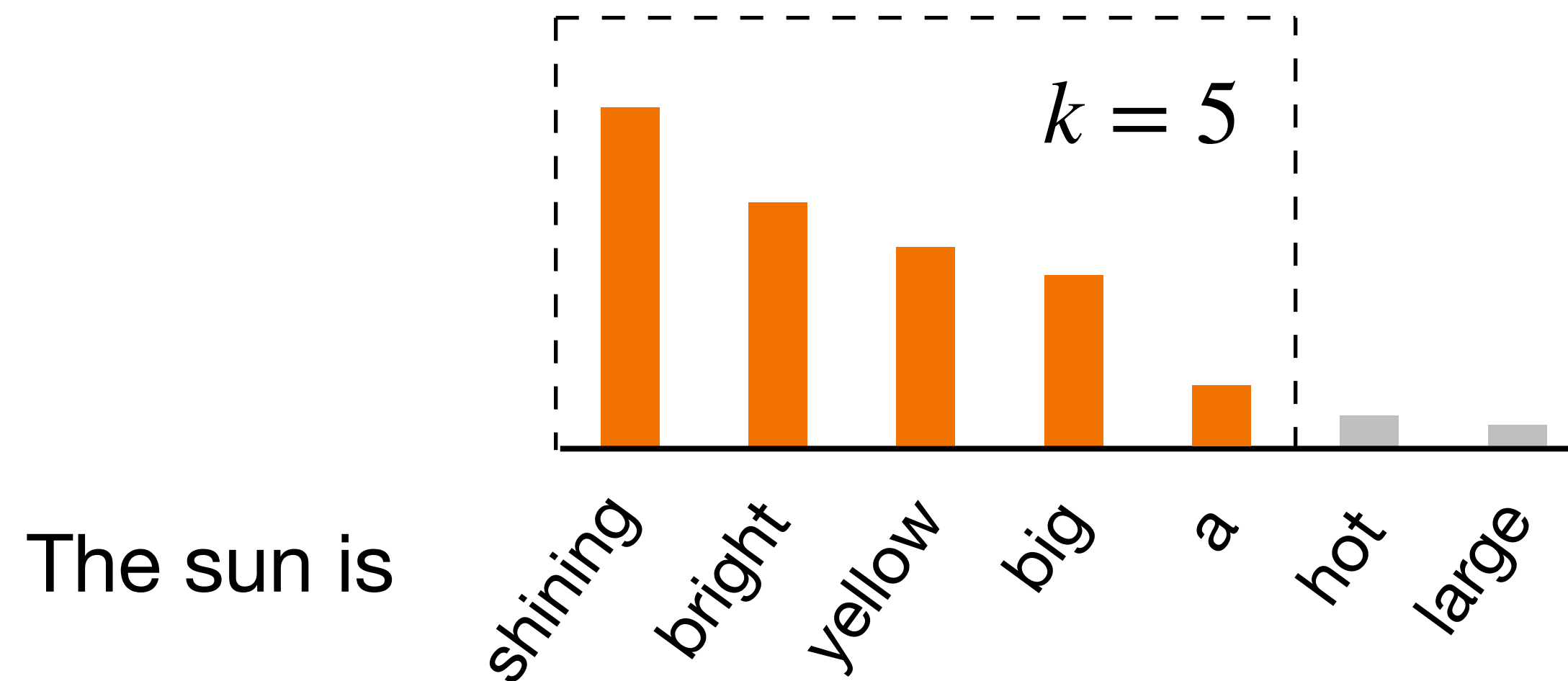
$p_1(y_t | y_{<t})$



$p_{0.5}(y_t | y_{<t})$

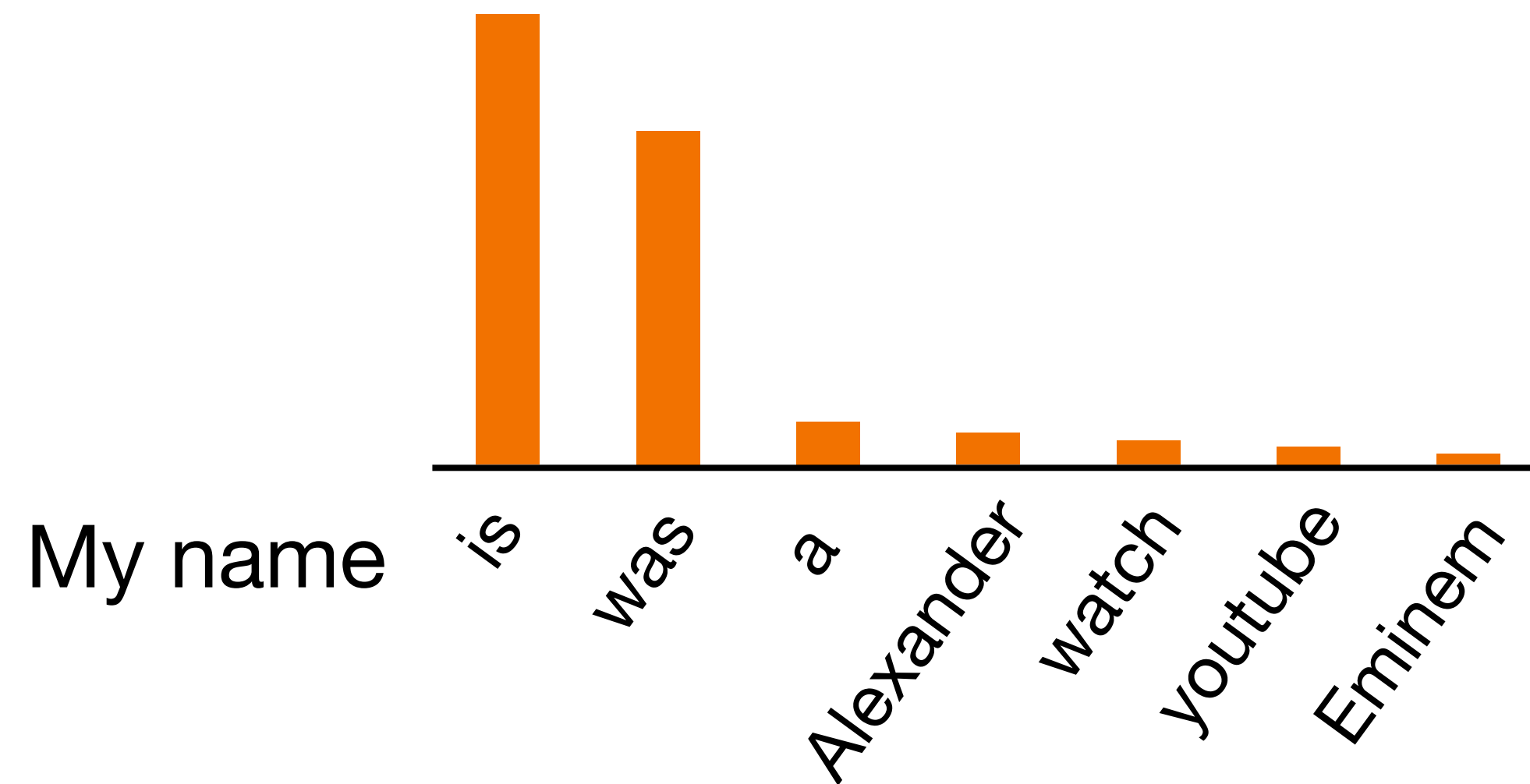
Top-k sampling

- Even with temperature, non-zero generation remains possible to produce a random token.
- We'll keep only the most probable tokens and sample from them.

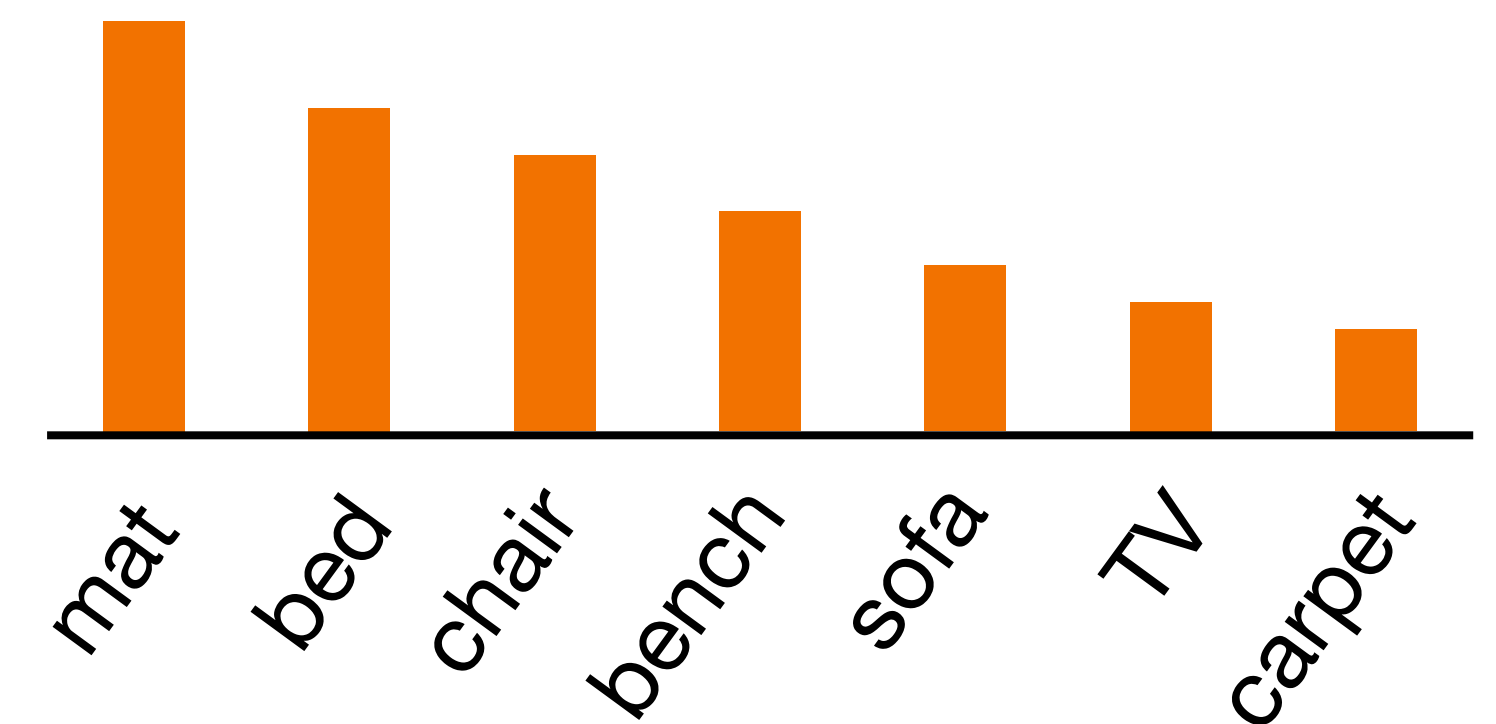


Top-k sampling

- Not all prefixes have the same number of most probable next tokens
- Because of this, it's impossible to find the ideal one k



Cat sits on a



Top-p sampling

Nucleus sampling

- We will select from the minimum number of tokens whose total probability is greater than p

$$\sum_{w \in V^{(p)}} p(w | y_{<t}) \geq p$$

$$|V^{(p)}| \rightarrow \min$$

A common value for p is 0.9 or 0.95